Adson Emanuel Santos Amaral

Desenvolvimento de um sistema supervisório para monitoramento contínuo de sinais

Natal – RN

Janeiro de 2025

Adson Emanuel Santos Amaral

Desenvolvimento de um sistema supervisório para monitoramento contínuo de sinais

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Andrés Ortiz Salazar

Universidade Federal do Rio Grande do Norte – UFRN

Departamento de Engenharia de Computação e Automação – DCA

Curso de Engenharia de Computação

Natal – RN

Janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Amaral, Adson Emanuel Santos.
Desenvolvimento de um sistema supervisório para monitoramento contínuo de sinais / Adson Emanuel Santos Amaral. – 2025.
84f.: il.

Monografia (Graduação) – Universidade Federal do Rio Grande do Norte, Centro de Tecnologia (CT), Engenharia de Computação, Natal, 2025.
Orientação: Dr. Andres Ortiz Salazar.

1. Sinais – Monografia. 2. DSP – Monografia. 3. Supervisório – Monografia. 4. SCI – Monografia. 5. PyQt5 – Monografia. I. Salazar, Andres Ortiz. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira – CRB-15/429

Adson Emanuel Santos Amaral

Desenvolvimento de um sistema supervisório para monitoramento contínuo de sinais

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Andrés Ortiz Salazar

Trabalho aprovado. Natal – RN, 17 de Janeiro de 2025:

Prof. Dr. Andrés Ortiz Salazar - Orientador
UFRN

Dr. Diego Antônio de Moura Fonsêca
UFRN

Me. Rodrigo de Andrade Teixeira
UFRN

Natal – RN
Janeiro de 2025

AGRADECIMENTOS

Agradeço a Deus, pela dádiva da vida e por ter me dado forças em todos os momentos em que pensei em desistir.

Aos meus pais Eliene Generosa dos Santos Amaral e Admilson Fernandes do Amaral, pelo amor incondicional e apoio incessante. Suas presenças foram fundamentais para minha formação pessoal e acadêmica.

Aos meus familiares, que sempre me incentivaram e nunca mediram esforços em me ajudar. Em especial, aos meus tios Luzinente Generosa e Miguel José, por terem me acolhido em sua casa e por todo o suporte que me deram durante a graduação.

À minha companheira Ana Paula de Oliveira, pela assistência inabalável e paciência constante.

Aos amigos que fiz durante a graduação. Sem os momentos de descontração e as várias horas de estudos em grupo que compartilhamos jamais teria conseguido chegar tão longe.

Ao meu orientador, Prof. Dr. Andrés Ortiz Salazar pelos ensinamentos e pela confiança depositada em mim desde o início da iniciação cientifica até este ponto.

Ao pessoal que faz parte do Laboratório de Avaliação de Medição em Petróleo (LAMP), pela colaboração, troca de conhecimentos e companheirismo.

RESUMO

A visualização de sinais em tempo real é um fator crucial quando se deseja entender o comportamento de sistemas, uma vez que, a análise de dados instantâneos permite que o operador detecte padrões e tome decisões de maneira mais rápida e oportuna. Nesse contexto, este trabalho apresenta o desenvolvimento de um sistema supervisório para sinais digitais. A fim de atingir esse objetivo, fez-se uso do Processador Digitais de Sinais (DSP) modelo TMS320F28335 para enviar dados via Interface de Comunicação Serial (SCI) para um computador a parte. Em seguida os dados são interpretados e exibidos utilizando-se a biblioteca PyQT5.

Palavras-chaves: Sinais; DSP; Supervisório; SCI; PyQt5.

ABSTRACT

The visualization of real-time signals is a crucial factor in understanding system behavior, as the analysis of instantaneous data enables operators to detect patterns and make decisions more quickly and effectively. In this context, this work presents the development of a supervisory system for digital signals. To achieve this goal, a Digital Signal Processor (DSP) model TMS320F28335 was utilized to send data via a Serial Communication Interface (SCI) to an external computer. The data is then processed and displayed using the PyQt5 library.

Keywords: Signals; DSP; Supervisory; SCI; PyQt5.

LISTA DE ILUSTRAÇÕES

Figura 1 – Estrutura básica da comunicação de dados. 16
Figura 2 – Modos do fluxo de dados. 17
Figura 3 – Conexões físicas nas comunicações Parelela e Serial. 18
Figura 4 – Tranmissão de um frame no formato 8-N-1. 20
Figura 5 – DSP 28335. 23
Figura 6 – Diagrama de Multiplexação das interrupções usando o bloco PIE. . . . 24
Figura 7 – Diagrama de blocos do módulo SCI. 26
Figura 8 – Amostragem dos bits no modo assíncrono do SCI. 27
Figura 9 – Formato de dados no modo multiprocessador de linha ociosa. 28
Figura 10 – Formato de dados no modo multiprocessador de bit de endereço. 28
Figura 11 – Ambiente de Desenvolvimento Code Composer Studio. 29
Figura 12 – Módulo Conversor FTDI FT232RL. 30
Figura 13 – Montagem dos dispositivos do projeto. 32
Figura 14 – Fluxograma do algoritmo implementado no DSP. 33
Figura 15 – *Frame* do caractere 5. 38
Figura 16 – *Frame* do caractere 5. 38
Figura 17 – Tela inicial do Qt *Designer*. 39
Figura 18 – Tela inicial do supervisório. 41
Figura 19 – Tela 41
Figura 20 – Exibição no tempo de um sinal de 2 Hz. 45
Figura 21 – Exibição no tempo de um sinal de 60 Hz. 46
Figura 22 – Exibição no tempo de um sinal de 60Hz com zoom horizontal aplicado. 46
Figura 23 – Exibição no tempo de dois sinais de 60 Hz defasados em 120°. 47
Figura 24 – Exibição no tempo de dois sinais de 60Hz defasados em 120° com zoom horizontal aplicado. 47
Figura 25 – Exibição no formato de Lissajous. 48
Figura 26 – Exibição no tempo de um sinal de corrente com zoom de 50% aplicado 49
Figura 27 – Sinal de corrente medido pelo osciloscópio. 49
Figura 28 – Exibição no tempo de dois sinais de corrente. 50
Figura 29 – Dois sinais de corrente medidos pelo osciloscópio. 51

LISTA DE TABELAS

Tabela 1 – Tabela ASCII dos caracteres utilizados. 31

LISTA DE ABREVIATURAS E SIGLAS

BPS	*Bits por segundo*
CCS	*Code Composer Studio*
CPU	*Central Processing Unit*
DSC	*Digital Signal Controller*
DSP	*Digital Signal Processor*
FTDI	*Future Technology Devices Internationa*
IRS	*Interrupt Service Routine*
I2C	*Inter-Integrated Circuit*
LAMP	Laboratório de Avaliação de Medição em Petróleo
NRZ	*Non-Return-to-Zero*
PIE	*Peripheral Interrupt Expansion*
RPM	*Rotações por Minuto*
SCI	*Serial Communication Interface*
SPI	*Serial Peripheral Interface*
TTL	*Transistor–Transistor Logic*
USB	*Universal Serial Bus*
UART	*Universal Asynchronous Receiver/Transmitter*

LISTA DE SÍMBOLOS

k	Prefixo quilo (10^3)
M	Prefixo Mega (10^6)
n	Prefixo nano (10^{-9})
μ	Prefixo micro (10^{-6})

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Contextualização . . . 13
1.2 Objetivos . . . 13
1.2.1 Objetivos específicos . . . 14
1.3 Estrutura do Trabalho . . . 14

2 FUNDAMENTAÇÃO TEÓRICA . . . 15
2.1 *Comunicação de dados* . . . 15
2.1.1 Componentes . . . 15
2.1.2 Fluxo de dados . . . 16
2.1.3 Transmissão digital de dados . . . 17
2.1.4 Taxas de Transmissão . . . 19
2.2 *Interface UART* . . . 19
2.2.1 Processo de Comunicação . . . 19
2.3 *Sistema Supervisório* . . . 21

3 FERRAMENTAS E TECNOLOGIAS UTILIZADAS . . . 22
3.1 *Processador Digital de Sinais* . . . 22
3.1.1 Sistema de Interrupção . . . 23
3.1.2 Interface de Comunicação Serial - SCI . . . 25
3.1.3 Code Composer Studio - CCS . . . 28
3.2 Módulo de conversão USB para TTL . . . 29
3.3 ASCII . . . 30
3.4 PyQt5 - Comprehensive Python Bindings for Qt v5 . . . 31

4 METODOLOGIA . . . 32
4.1 Montagem . . . 32
4.2 Programação do DSP28335 . . . 32
4.2.1 Configuração da Interface de Comunicação Serial . . . 34
4.2.2 Função para conversão de tipo de dados . . . 35
4.2.3 Funções para envio de dados . . . 36
4.2.4 Análises do tempo de envio . . . 36
4.3 Desenvolvimento do supervisório . . . 39
4.3.1 Interface do Supervisório . . . 40
4.3.2 Estrutura da lógica da aplicação . . . 42

5 RESULTADOS 45
5.1 Testes simulados 45
5.2 Testes com motor em Operação 48

6 CONCLUSÃO 52

REFERÊNCIAS 53

APÊNDICES 55

APÊNDICE A – CÓDIGO-FONTE DO DSP 56

APÊNDICE B – CÓDIGO-FONTE DA INTERFACE GRÁFICA DO SUPERVISÓRIO 63

APÊNDICE C – CÓDIGO-FONTE DA APLICAÇÃO 73

1 INTRODUÇÃO

O processamento digital de sinais é um ramo da ciência da computação que trata da matemática, dos algoritmos e das técnicas utilizadas para manipular sinais que geralmente foram convertidos para o formato digital. (YU et al., 2011).

Os dispositivos projetados para lidar especificamente com a manipulação de sinais são chamados de Processadores Digitais de Sinal (do inglês, *Digital Signal Processors* - DSP's) e podem ser aplicados em diversas áreas como: comunicação, medicina, geofísica e engenharia.

Durante a execução de um sistema que utiliza o DSP, o processamento de sinais ocorre de forma sequencial e frequentemente em alta velocidade, o que dificulta a análise em tempo real do impacto desses sinais no desempenho e no comportamento geral do sistema.

Nesse contexto, sistemas supervisórios apresentam-se como ferramentas relevantes ao viabilizar o monitoramento contínuo e eficiente dos sinais, sendo uma abordagem que supera não só as limitações das análises *offline*, mas também contribui para uma maior precisão e confiabilidade na tomada de decisões.

Dessa forma, este trabalho propõe o desenvolvimento de um sistema supervisório capaz de interpretar sinais digitais provenientes do DSP e exibi-los em gráficos, utilizando a comunicação serial do DSP para a transmissão dos dados durante a execução dos programas.

1.1 Contextualização

Este trabalho foi desenvolvido em parceria com o Laboratório de Avaliação de Medição em Petróleo (LAMP) da Universidade Federal do Rio Grande do Norte - UFRN. O LAMP possui pesquisas vinculadas a área de petróleo, energias renováveis e máquinas elétricas, onde na última área citada faz-se uso de DSP's. Desse modo, o estudo de caso ao qual será aplicado o projeto será feito neste laboratório.

1.2 Objetivos

O projeto proposto apresenta dois objetivos gerais. O primeiro, diz respeito ao envio de dados do DSP via Interface de Comunicação Serial - SCI, sejam eles gerados a partir de cálculos internos ao microcontrolador ou resultados do processamento de conversões

analógicas digitais. O segundo, está relacionado a criação de um supervisório que irá rodar em uma máquina, doravante chamada de computador de supervisão. Desse modo, o supervisório deve ser capaz de interpretar os dados que chegam ao computador de supervisão e exibi-los em gráficos previamente configurados pelo usuário.

1.2.1 Objetivos específicos

Os objetivos específicos são:

- Configurar a SCI do processador digital de sinais;
- Integrar a um código já existente a funcionalidade de envio de dados utilizando SCI;
- Definir uma forma de converter informações seriais TTL (Transistor–Transistor Logic) em sinais seriais USB (Universal Serial Bus);
- Desenvolver um código em Python capaz de ler a porta USB do computador e transmitir as informações processadas à interface de supervisão;
- Desenvolver um supervisório que permita ao usuário definir a quantidade de curvas a serem exibidas (uma ou duas) e o formato do gráfico (no tempo ou Lissajous).
- Garantir que o sistema supervisório seja capaz de apresentar corretamente sinais com frequência mínima de 60 Hz.

1.3 Estrutura do Trabalho

Este trabalho apresenta uma introdução sobre o processador digital de sinais, mostrando a importância de um sistema supervisório nos contextos em que esse dispositivo é utilizado. Em sequência, o Capítulo 2 apresenta a teoria que embase o desenvolvimento da pesquisa, ao passo que o Capítulo 3 aborda as ferramentas e tecnologias que foram utilizadas para implementação do projeto. O Capítulo 4, por sua vez, explica a metodologia aplicada para estruturar os códigos do DSP e do supervisório. O Capítulo 5 contém os resultados obtidos durante a implementação do projeto. Por fim, o Capítulo 6 traz as principais conclusões deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo é dedicado a apresentar os principais conceitos nos quais se baseia o projeto, sendo dividido em três seções distintas. Inicialmente, será apresentada a seção sobre comunicação de dados, que discutirá alguns princípios fundamentais para a troca de informações entre sistemas. Em seguida, a seção sobre UART explorará as características e o funcionamento dessa interface de comunicação. Por fim, a seção sobre sistemas supervisórios abordará o que são esses sistemas e como eles se aplicam a este trabalho.

2.1 *Comunicação de dados*

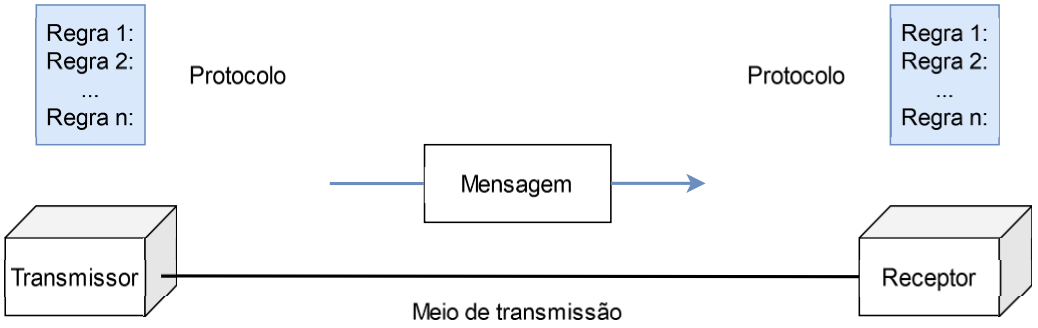
A comunicação de dados refere-se a troca de informações entre dispositivos através de algum meio de transmissão. Para que ela ocorra, é necessário que os dispositivos façam parte de um sistema de comunicação, que consiste em uma combinação de *Hardware* e *Software*. Forouzan (2013) afirma que a eficiência desses sistemas depende de quatro características essenciais:

- **Entrega:** o sistema deve entregar os dados ao destino correto, fazendo com que eles sejam recebidos apenas pelo dispositivo ou usuário pretendido;
- **Precisão:** é necessário que o sistema entregue os dados de forma precisa, tornando-se inutilizáveis dados que forem alterados durante a transmissão e não corrigidos;
- **Pontualidade:** os dados devem ser entregues pelo sistema de maneira pontual. Entregas tardias comprometem sua utilidade;
- ***Jitter:*** diz respeito à variação no tempo de chegada dos pacotes, caracterizada pelo atraso desigual na entrega de dados de áudios e vídeo;

2.1.1 Componentes

A estrutura básica de um sistema de comunicação de dados é composta por 5 componentes e pode ser vista na Figura 1.

Figura 1 – Estrutura básica da comunicação de dados.



Fonte: Elaborada pelo autor.

A mensagem, disposta como elemento central na Figura 1, diz respeito a informação que se deseja transmitir e pode possuir diversos formatos, como: textos, números, imagens, áudio e vídeo. O dispositivo responsável por enviá-la é chamado de remetente ou transmissor, ao passo que o dispositivo encarregado de recebê-la é nomeado destinatário ou receptor.

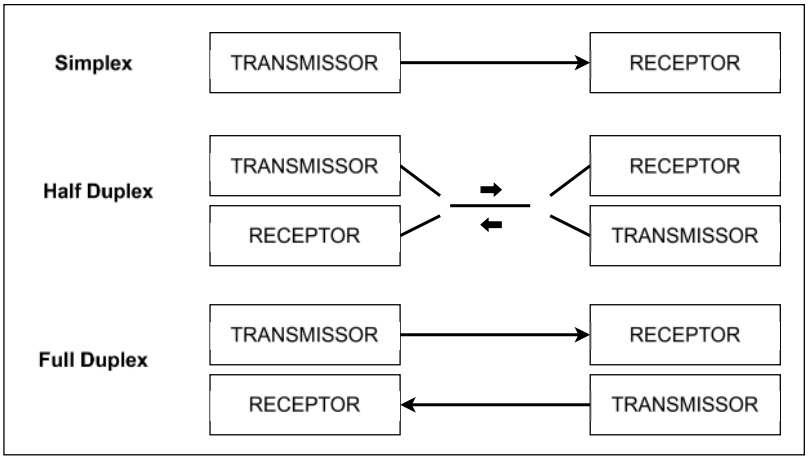
Interligando transmissor ao receptor, tem-se o meio ou linha de transmissão, sendo este o caminho físico por onde a mensagem se desloca. Alguns exemplos desse componente são o cabo coaxial e o fio par trançado.

Por fim, para que a mensagem enviada seja compreendida corretamente pelo destinatário, é necessário que haja um protocolo na comunicação. Este elemento estabelece um conjunto de regras que os dispositivos devem seguir para se comunicarem de maneira correta.

2.1.2 Fluxo de dados

O fluxo de dados refere-se ao movimento de informações entre elementos que compõem a comunicação e pode ocorrer em três modos: simplex, half-duplex e full-duplex. A Figura 2 mostra como são realizadas as conexões em cada um deles.

Figura 2 – Modos do fluxo de dados



Fonte: Elaborada pelo autor.

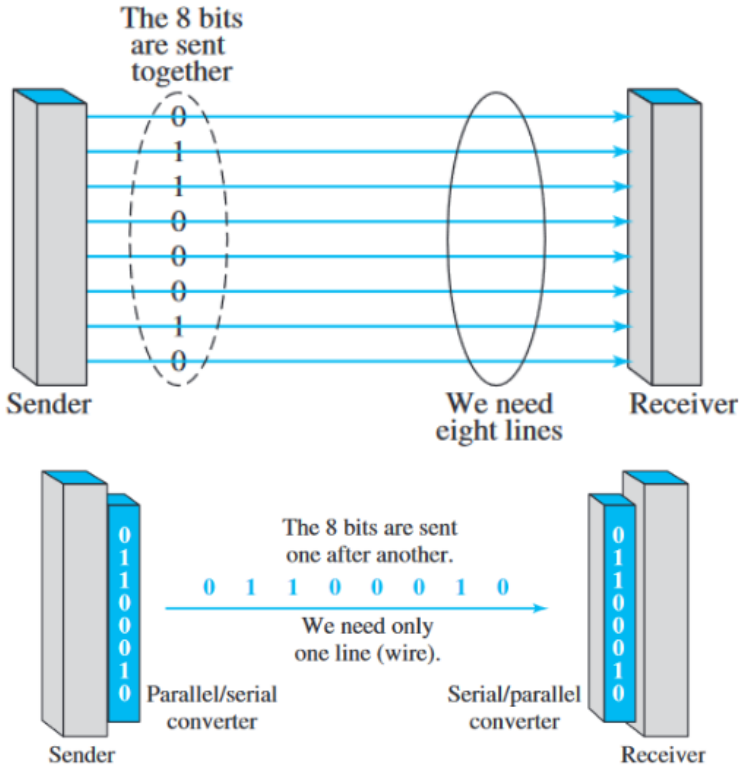
No modo *simplex* a comunicação é feita em um único sentido e somente um dos dois dispositivos pode transmitir dados, enquanto o outro apenas recebe. Já no modo *half-duplex*, cada dispositivo pode transmitir e receber dados, desde que essas tarefas não ocorram ao mesmo tempo, logo, quando um está enviando o outro está recebendo e vice-versa. No modo *full-duplex*, os dois dispositivos podem enviar e receber dados simultaneamente (FOROUZAN, 2013).

2.1.3 Transmissão digital de dados

A transmissão digital de dados é o envio de informações em formato de dados digitais, que são representados por sequências de bits 0's e 1's. Ela pode ser realizada em dois modos: paralelo ou serial.

Para Forouzan (2013), a transmissão paralela é um método em que bits são enviados por n canais de comunicação a cada pulso de *clock*, enquanto na transmissão serial um bit segue o outro, sendo necessário apenas um canal. A Figura 3 ilusta como são feitas as ligações para uma transmissão paralela com 8 canais e para a transmissão serial.

Figura 3 – Conexões físicas nas comunicações Parelela e Serial



Fonte: (FOROUZAN, 2013)

Ainda segundo Forouzan (2013), em termos de usabilidade, as principais diferenças entre os dois modos de transmissão de dados são a velocidade e o custo. A transmissão paralela possui a capacidade de aumentar a velocidade de transferência em um fator de n em relação à transmissão serial. Porém, essa abordagem vem acompanhada de um alto custo de transmissão, uma vez que são necessários mais canais apenas para enviar o fluxo de dados. Desse modo, a comunicação paralela geralmente é limitada a curtas distâncias, enquanto a comunicação serial é mais eficaz em longas distâncias.

Além de apresentar redução nos curtos e simplicidade, a comunicação serial ocorre de duas maneiras que influenciam diretamente sua eficiência e aplicabilidade, sendo elas: comunicação serial síncrona e assíncrona.

Conforme Axelson (2007), em um protocolo síncrono, a interface possui uma linha de *clock* controlada por um dos dispositivos e todos os bits transmitidos são sincronizados com esse *clock*. Assim, a transmissão de cada um deles é tida como válida em um instante específico após a transição de subida ou descida do sinal da linha. Algumas interfaces seriais síncronas são a I2C (*Inter-Integrated Circuit*), SPI (*Serial Peripheral Interface*) e *Microwire*.

Já sobre o protocolo assíncrono, Axelson (2007) aponta que não há a presença de

uma linha de transmissão específica para compartilhamento do *clock* que sincroniza a troca de dados, em vez disso, uma frequência é acordada entre os dispositivos que participam da comunicação e cada um utiliza seu próprio *clock* como referência de tempo. Dessa forma, a sincronia entre os dispositivos é feita a partir do envio de um bit específico chamado bit de início. Dentre os exemplos de comunicação assíncrona existem a UART e RS-232.

2.1.4 Taxas de Transmissão

As taxas de transmissão determinam a velocidade com que ocorre a transferência de dados. Desse modo, a literatura aborda dois conceitos importantes envolvendo essas taxas: o *bit rate* e o *baud rate*.

Segundo Axelson (2007), *bit rate* é o número de bits que são transmitidos ou recebidos por unidade de tempo, sendo geralmente expresso em bits por segundo, enquanto o *baud rate*, diz respeito ao número de transições de sinais por segundo.

Neste trabalho, cada transição de dados representa a mudança de um bit, fazendo com que *bit rate* e *baud rate* sejam os mesmos. Portanto, o termo adotado pra se referir a taxa de transmissão será *baud rate*.

2.2 *UART*

A UART (*Universal Asynchronous Receiver/Transmitter*) é uma interface de comunicação serial assíncrona que define um protocolo para a troca de dados entre dispositivos de maneira sequencial, bit a bit.

Os elementos que a compõem são um transmissor (TX), um receptor (RX) e um fio de terra. A conexão física entre eles é feita com o transmissor de um dispositivo conectado ao receptor do outro (assim como mostrado no formato *Full Duplex* da subseção 2.1.2) e com um aterramento comum.

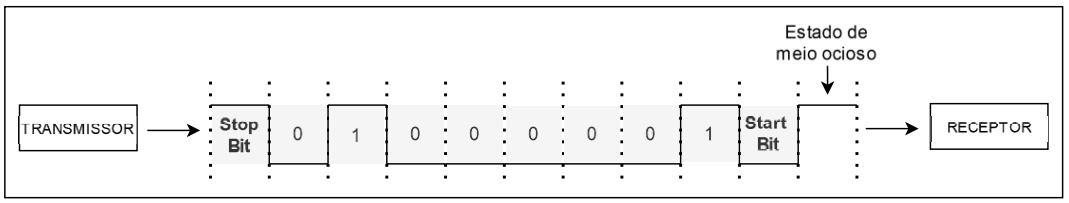
2.2.1 Processo de Comunicação

Tendo em vista a característica assíncrona da UART, as informações transmitidas seguem um padrão de formatação que as dividem em blocos geralmente chamados de *frames*. Cada *frame* é formado por: um bit de início, bits de dados (que podem ser chamados de palavra), um bit opcional de paridade e um ou mais bits de parada.

A partir da quantidade de bits configuráveis, existem diversos tipos de formatos para os *frames*. Em um dos mais comuns, o 8-N-1, o *frame* possui oito bits de dados, nenhum bit de paridade e um bit de parada. Tal formato tem seus níveis lógicos em uma

linha de transmissão retratados na Figura 4 e será utilizado para ilustrar como funciona o processo de comunicação UART.

Figura 4 – Tranmissão de um frame no formato 8-N-1.



Fonte: Elaborada pelo autor.

Quando não está ocorrendo comunicação, a linha de transmissão encontra-se em estado de meio ocioso com o transmissor em nível lógico alto (1). Para indicar o início de uma transmissão, o transmissor envia um sinal de referência conhecido como bit de início, que possui nível lógico baixo (0) e é responsável por alertar ao receptor sobre a chegada de um novo *frame*.

A partir da chegada do bit de início, o receptor tenta amostrar os bits seguintes considerando o período do bit, que diz respeito ao tempo que se leva para transmitir um único bit e matematicamente pode ser representado como o inverso do *baud rate*.

Em seguida, são enviados 8 bits de dados em sequência começando pelo bit menos significativo. Os bits de dados contém a informação útil que se deseja transmitir e sua quantidade pode variar de 5 a 8, a depender do dispositivo ao qual a UART está vinculada.

Após os bits de dados, o transmissor envia o bit de parada para finalizar a transmissão. Este bit possui valor lógico alto e pode ser de uma, uma e meia ou duas vezes o tempo do período do bit.

Além dos bits explicitados, é importante comentar sobre o bit de paridade, que não foi utilizado no exemplo em questão. Tal bit, se configurado, atua como uma forma simples de verificação de erros, sendo calculado pelo transmissor com base na palavra e adicionado ao *frame* depois desta e antes do bit de parada.

Desse modo, existem quatro tipos de paridade: par (*even*), ímpar (*odd*), de marca e de espaço, representadas nos formatos de *frames* respectivamente pelas letras "E", "O", "M" e "S". Em contextos onde a paridade é definida como par ou ímpar, o número total de bits "1" no frame tem que ser par para o primeiro caso e ímpar para o segundo. Assim, o bit paridade é configurado como 0 ou 1 para fazer com que o valor total de bits atinja a paridade desejada. Nos casos em que a paridade é definida como de marca, o bit de paridade é sempre 1 e quando é definida como de espaço, o bit de paridade é sempre 0. Quando um *frame* que possui a paridade configurada chega ao receptor, este verifica se ela está correta para identificar se houveram ou não erros na transmissão.

Ademais, para que a comunicação UART funcione corretamente, transmissor e receptor precisam concordar a respeitos do *baud rate* e do formato do *frame*. Caso contrário, a sincronização entre os dispositivos pode ser comprometida e a interpretações dos dados feita de maneira errônea.

2.3 *Sistema Supervisório*

Um sistema computacional considerado como supervisório opera em torno de dados, possuindo como características: a capacidade de estabelecer comunicação entre computadores e equipamentos de aquisição, a manipulação e análise dos dados e a exibição dos resultados destes processamentos ao usuário, em um formato previamente definido (TIBOLA, 2000).

Os sistemas supervisórios fornecem uma interface visual amigável para visualização de variáveis do sistema em tempo real, permitindo que o operador compreenda e atue sobre o processo de maneira facilitada.

No presente trabalho, o supervisório desenvolvido apresenta as seguintes características: suporte aos protocolos de comunicação utilizados pelo DSP; mecanismos que buscam assegurar a sincronia, de modo que os dados exibidos representem o estado atual do processo; e possibilidade de ajustes dinâmicos na visualização dos parâmetros monitorados.

3 FERRAMENTAS E TECNOLOGIAS UTILIZADAS

Este capítulo contém as descrições do componentes eletrônicos e do software utilizados para desenvolvimento e uso do supervisório. Inicialmente é apresentada uma visão geral do Processador Digital de Sinais e de seus periféricos empregados. Em seguida, é abordado o módulo de conversão UART para USB. Por fim, é discutido o PyQt5, destacando suas funcionalidades e programação.

3.1 *Processador Digital de Sinais*

Neste trabalho, o termo Processador Digital de Sinais, refere-se a um conjunto composto por um Controlador Digital de Sinais (do inglês, *Digital Signal Controller* - DSC) e sua placa de desenvolvimento.

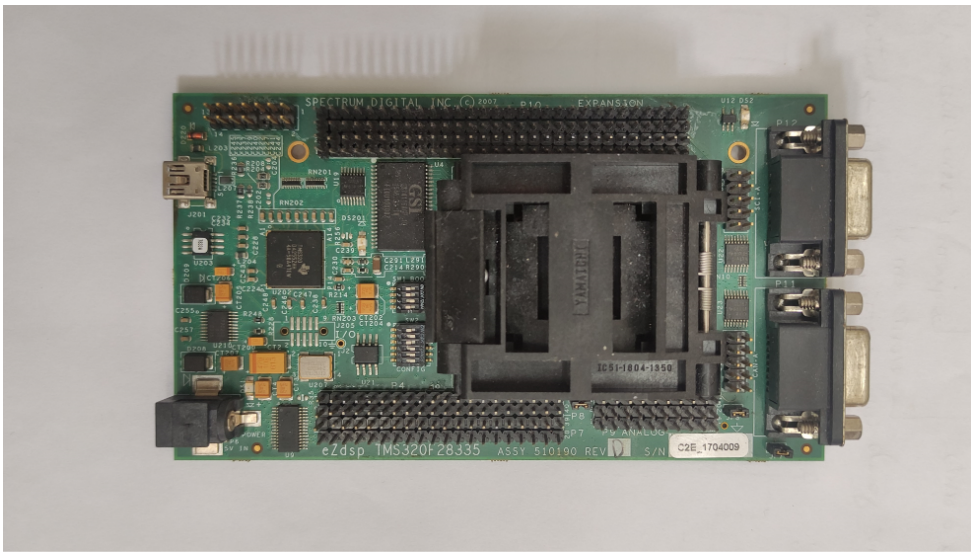
O modelo de DSC utilizado é o TMS320F28335. Ele faz parte da família $C2000^{TM}$ de microcontroladores em tempo real desenvolvida pela Texas Instruments, sendo projetado para atuar em sistemas que precisem de controle rápido e preciso.

O módulo *stand-alone* associado ao TMS320F28335 é o $eZdsp^{TM}F28335$ fabricado pela Spectrum Digital. Esta placa permite o acesso facilitado a pinos, o que simplifica a conexão de periféricos e sensores; e oferece recursos como conectores de alimentação e interfaces de comunicação.

Nesse viés, o DSP28335 (forma como, por simplicidade, será chamado no texto o conjunto descrito) pode ser visto na Figura 5 e apresenta como características principais conforme Instruments (2007):

- Chip de alta performance com unidade de ponto flutuante de 32 bits;
- Frequência de operação de 150MHz (ciclo de *clock* 6,67ns);
- Consumo de energia de 1,9V/1,8V para o processador e 3,3V para dispositivos de entrada e saída;
- Interface JTAG USB embutido;
- Memórias RAM de 68k bytes, Flash de 512k bytes e SRAM de 256k bytes;
- Conversor analógico-digital (A/D) de 12 bits no chip com 16 canais de entrada;
- Relógio de entrada de 30 MHz;

Figura 5 – DSP 28335.



Fonte: Elaborada pelo autor.

Para a realização do projeto, foi necessário implementar o envio controlado de sinais para outro dispositivo, assegurando que não houvesse interferência no processamento desses sinais. Os periféricos do DSP28335 utilizados para atender a essas requisitos foram: o Sistema de Interrupção e o Módulo SCI. Além disso, fez-se uso do *Code Composer Studio* (CCS) versão 3.3 para realizar a programação e depuração do código fonte desenvolvido.

As subseções a seguir contém uma descrição detalhada a respeito da configuração e do funcionamento desses periféricos e do CCS.

3.1.1 Sistema de Interrupção

A interrupção é um mecanismo em que o processador tem a execução do seu *loop* principal interrompida para lidar com um evento específico sem a necessidade de se verificar constantemente a ocorrência desse evento. Quando uma interrupção é inicializada, o controle da execução do programa atual é transferido para uma rotina de tratamento de interrupção (também conhecida como ISR - *Interrupt Service Routine*), que se encarrega de identificar qual rotina foi ativada e executar as instruções associadas a ela.

Nesse viés, o núcleo de DSP 28335 suporta trabalhar com interrupções, podendo atender a uma interrupção não mascarável (NMI) e a 16 solicitações de interrupções mascaráveis (INT1-INT14, RTOSINT e DLOGINT), onde a interrupção não mascarável é um tipo de interrupção que não pode ser temporariamente desativada ou ignorada pelo sistema, ao passo que a mascarável pode, a depender de sua prioridade em relação as outras interrupções.

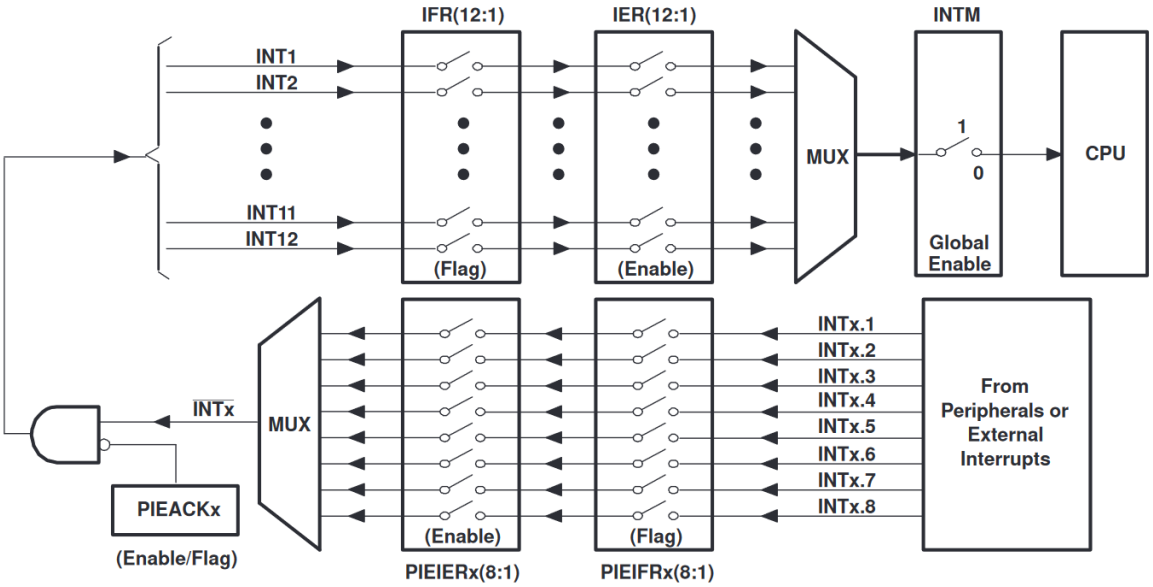
No entanto, como as interrupções podem ser geradas por diversas fontes, a Unidade

Central de Processamento (do inglês, *Central Processing Unit* - CPU) não possui capacidade suficiente para tratar todas as solicitações de interrupção em seu nível. Logo, as interrupções geradas por periféricos e outros pinos externos são gerenciadas por um bloco de expansão de interrupção de periféricos (*Peripheral Interrupt Expansion* - PIE).

O PIE realiza a multiplexação das várias fontes de interrupções em um conjunto menor de entrada de interrupções, dessa forma, ele organiza as 96 fontes presentes no DSP em 12 grupos, que são nomeados de grupo PIE 1 até o grupo PIE 12, e cada um desses grupos possui 8 possíveis fontes de interrupção. Para as interrupções não multiplexadas, o PIE passa a solicitação diretamente para a CPU.

A partir do diagrama da Figura 6, se faz possível esmiuçar o sequenciamento lógico de funcionamento do PIE.

Figura 6 – Diagrama de Multiplexação das interrupções usando o bloco PIE.



Fonte: Instruments (2020).

Tal sequenciamento é dividido em três níveis:

- Nível do Periférico: quando uma interrupção ocorre devido um evento em um periférico, um *bit flag* (IF) relacionado a esse evento é configurado em um registrador próprio para esse periférico. Se o *bit enable* (IE) correspondente está configurado, o periférico faz um pedido de interrupção ao controlador PIE, caso contrário, o *bit IF* fica ativo até ser reiniciado via *software*. Se por algum motivo o *bit enable* foi ativado um tempo depois e o bit IF ainda estiver configurado, o pedido de interrupção é enviado ao PIE;
- Nível PIE: cada grupo de interrupção no bloco PIE possui um registrador de *flag* (PIEIFRx) e um registrador *enable* (PIEIERx). Além disso, há um bit de

reconhecimento (PIEACKx) associado a cada grupo. Nos casos, o 'x' referencia o grupo, variando de 0 a 12, e o 'y' a interrupção, variando de 0 a 8. Quando uma solicitação de interrupção chega ao controlador PIE, o bit PIEIFRx,y correspondente é ativado. Se o bit PIEIERx,y estiver ativo para a mesma interrupção, o PIE verifica se o bit PIEACKx correspondente também está ativo. Se o PIEACKx estiver desativado para esse grupo, o PIE envia o pedido de interrupção para a CPU, caso contrário ele aguarda até ser desativado para enviar o pedido para INTx;

- Nível de CPU: quando uma solicitação de interrupção é enviada à CPU, o *bit de flag* de interrupção da CPU (IFR) correspondente à INTx é ativado. Então a interrupção correspondente só é atendida quando o registrador de habilitação de interrupção da CPU (IER) ou o registrador de habilitação de interrupção de depuração (DGBIER) estão corretamente habilitados, e o bit de máscara global de interrupção (INTM) está ativo.

Após chegar na CPU a solicitação de interrupção deve ser atendida, mas para isso se faz necessário saber qual ISR deve ser chamada. Dessa forma, cada uma das fontes de interrupção citada é identificada por um vetor individual que fica armazenado em um bloco de RAM (*Dynamic Random Acess Memory*), então há uma tabela de vetores PIE para mapear as ISR correspondentes às suas respectivas fontes.

Assim, quando uma interrupção é gerada o DSP faz uma consulta a tabela para determinar qual ISR deve ser executada.

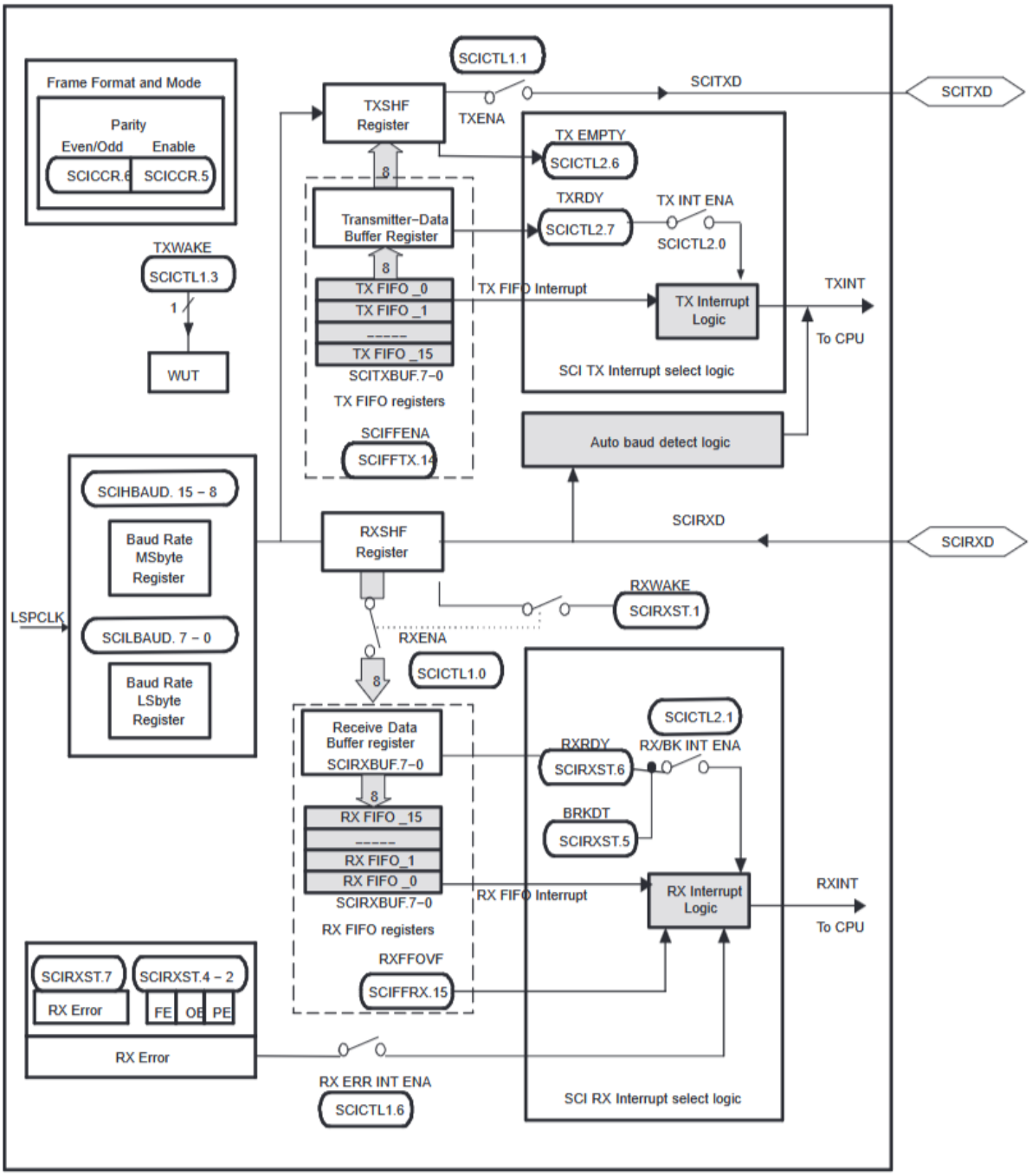
3.1.2 Interface de Comunicação Serial - SCI

O DSP28335 apresenta três módulos de Interface de Comunicação Serial - SCI (o SCI-A, SCI-B e SCI-C), sendo eles portas seriais assíncronas baseadas em UART, que podem ser programados para atuar com uma taxa de transmissão de até 64 mil valores.

A transmissão e recepção de dados efetuada por meio do SCI é feita a partir de pinos externos chamados respectivamente SCITXD e SCIRXD. Ambos possuem uma fila FIFO (*First In, First Out*) de 16 níveis de profundidade cada e têm seus próprios pinos bits de habilitação e interrupção, além de poderem operar no modo *half-duplex* ou *full-duplex*. A Figura 7 mostra o diagrama de blocos do módulo SCI, onde é possível observar blocos referentes às características descritas e demais funcionalidades, como a seleção de paridade.

Este periférico suporta comunicações digitais entre a CPU e periféricos assíncronos que usam o formato NRZ (*non-return-to-zero*). No NRZ, cada bit é representado por um nível de tensão constante, com o nível do sinal em alto quando o bit é "1" e em nível baixo quando é "0".

Figura 7 – Diagrama de blocos do módulo SCI.



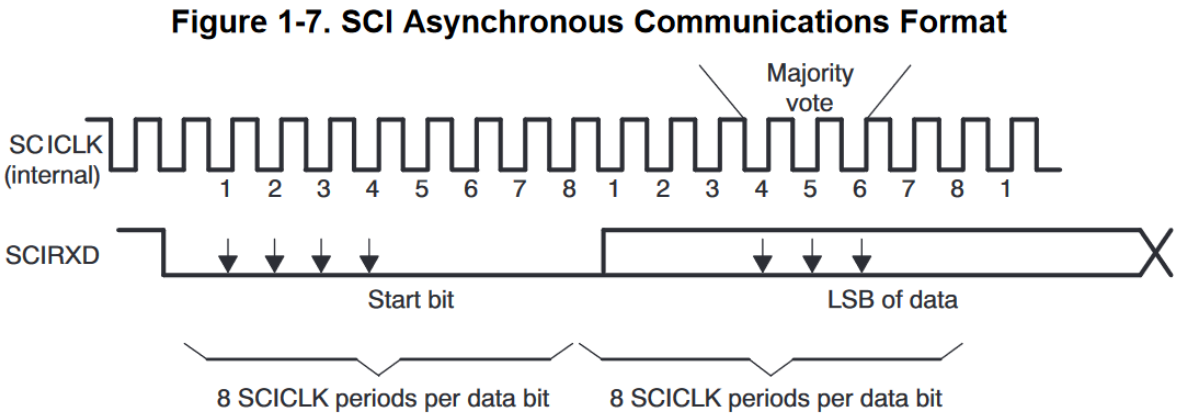
Fonte: Instruments (2008).

O formato de comunicação do SCI pode ser unidirecional ou bidirecional e os *frames* consistem em um bit de início, de um a oito bits de dados, um bit de paridade opcional (par ou ímpar) e um ou dois bits de parada.

A leitura dos bits depende diretamente do sinal de *clock* definido para o SCI, chamado SCICLK. O padrão é que existam 8 períodos de SCICLK por bit de dados, ou seja, o *clock* de recepção é 8 vezes superior a taxa de bits suportada.

Dessa maneira, considerando o processo de comunicação UART discutido na subseção 2.2.1, o receptor inicia a operação ao receber um bit de início válido, que segundo as especificações do módulo SCI, é identificado por quatro períodos seguidos de SCICLK com o bit em '0'. Caso alguma leitura não seja 0, o processador reinicia e começa a procurar por outro bit de início. A Figura 8 exemplifica como é feita essa amostragem dos bits é feita.

Figura 8 – Amostragem dos bits no modo assíncrono do SCI.



Fonte: Instruments (2008).

Para os bits que vem após o de início, o processador determina seus valores ao fazer a amostragem do sinal nos períodos intermediários de SCICLK, que são: quatro, cinco e seis. O valor final atribuído ao bit é baseado na maioria das três amostras.

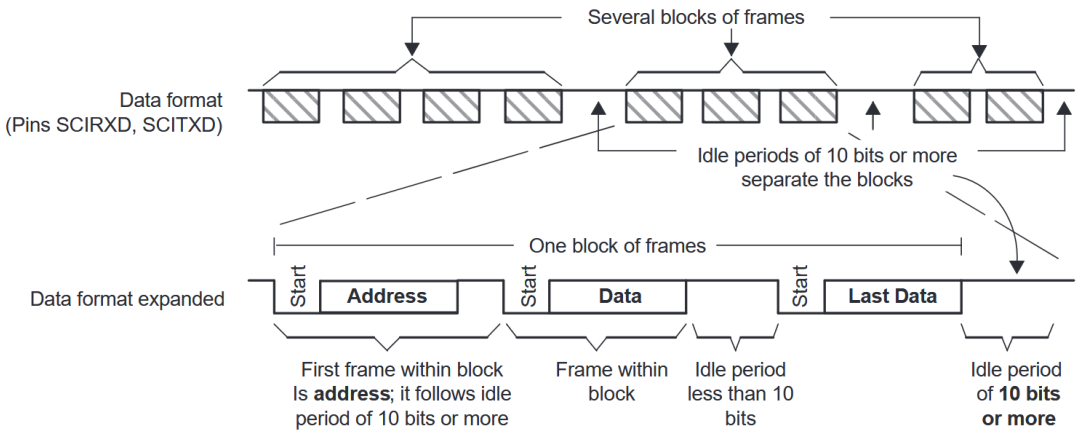
As várias verificações feitas nos bits durante a comunicação assíncrona buscam garantir a integridade da comunicação, minimizando erros, uma vez que a amostragem do bit em diferentes momentos utilizando o SCICLK, possibilita a identificação correta do valor do bit, mesmo que haja ruído ou flutuações rápidas no sinal.

Ademais, o módulo SCI oferece dois modos de operação visando flexibilidade para adaptar a comunicação a diversas configurações. Eles são: os modos multiprocessador de linha ociosa (*idle-line*) e de bit de endereço (*address-bit*), que como o nome sugere, permitem a comunicação entre processadores no mesmo *link* serial.

Nos modos multiprocessador, são criados blocos de *frames* que organizam os dados de forma estruturada, o que facilita a identificação e processamento das informações. O modo de linha ociosa é caracterizado por um período de ociosidade em que não há transmissão de dados. Dessa forma, os blocos são separados por um tempo ocioso mais longo do que os intervalos de ociosidade entre *frames* dentro de cada bloco, assim como mostrado na Figura 9, onde um tempo ocioso de 10 ou mais períodos do bit em nível alto indica o início de um novo bloco.

Já no modo multiprocessador de bit de endereço, um bit extra chamado bit de endereçamento é inserido nos *frames* após o último bit de dados. Ele é definido como 1 no

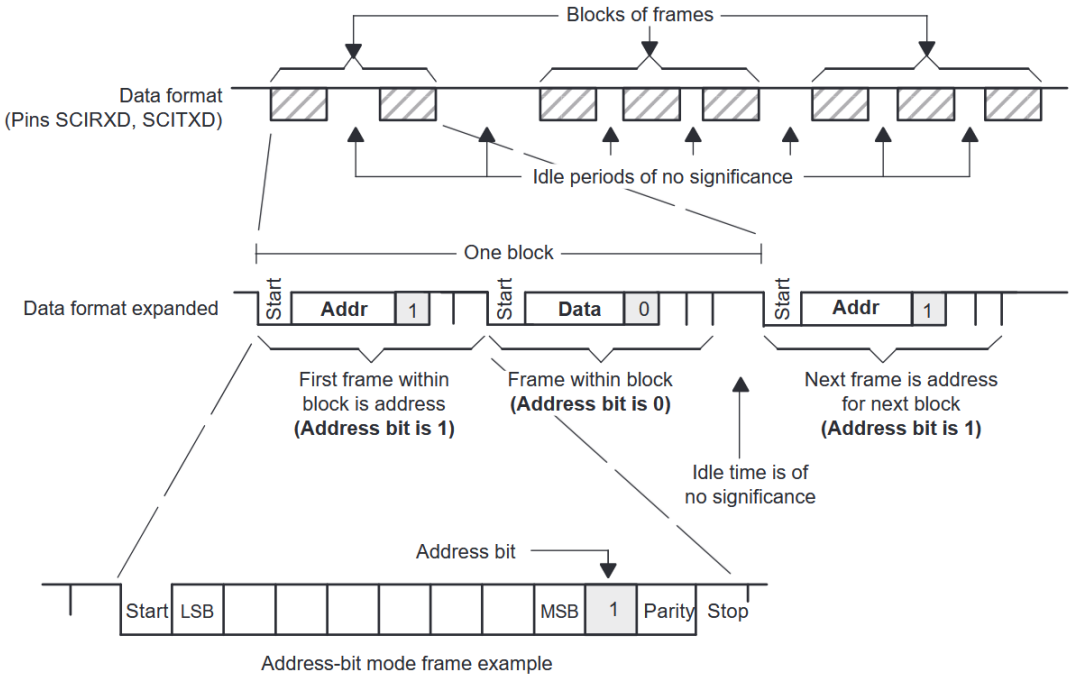
Figura 9 – Formato de dados no modo multiprocessador de linha ociosa.



Fonte: Instruments (2008).

primeiro *frame* do bloco e como 0 nos demais. A Figura 10 mostra como é o formato da comunicação nesse modo.

Figura 10 – Formato de dados no modo multiprocessador de bit de endereço.



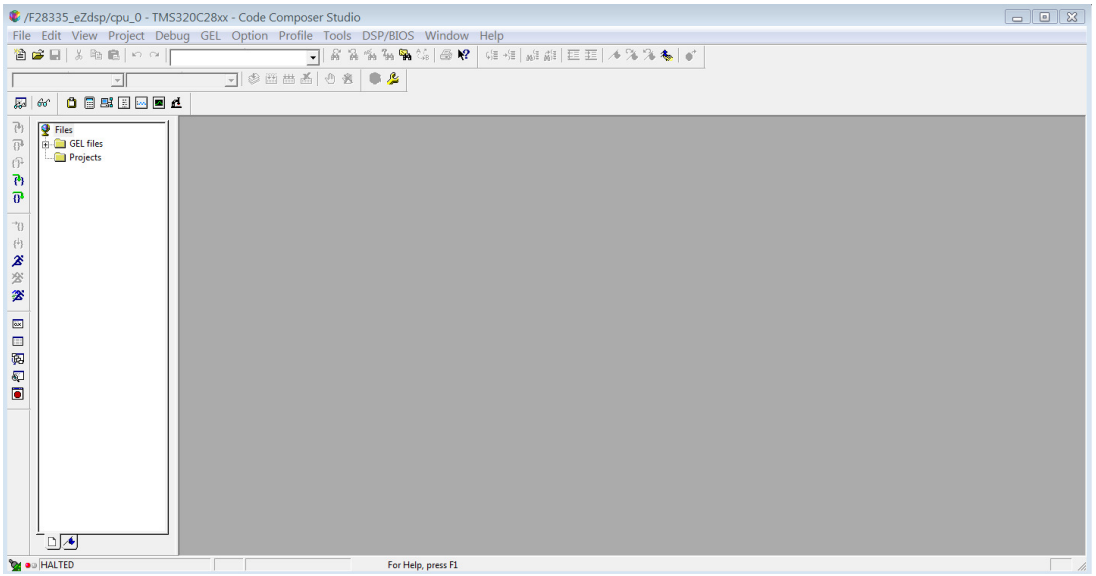
Fonte: Instruments (2008).

3.1.3 Code Composer Studio - CCS

O Code Composer Studio (CCS) é um ambiente de ambiente de desenvolvimento integrado (IDE) associado ao DSP 28335 e aos demais microcontroladores e processadores embarcados da *Texas Instruments*.

Ele oferece ferramentas úteis quando se deseja programar e gerenciar projetos, tais como: editor de código fonte, depurador e *profiler*. Além disso, possui suporte para programação nas linguagens C, C++ e Assembly. A Figura 11 mostra uma imagem do ambiente do *Code Composer Studio*.

Figura 11 – Ambiente de Desenvolvimento Code Composer Studio.



Fonte: Elaborada pelo autor.

Vale ressaltar que a interface de comunicação JTAG USB é responsável por promover a interação entre o DSP28335 e o CCS.

3.2 Módulo de conversão USB para TTL

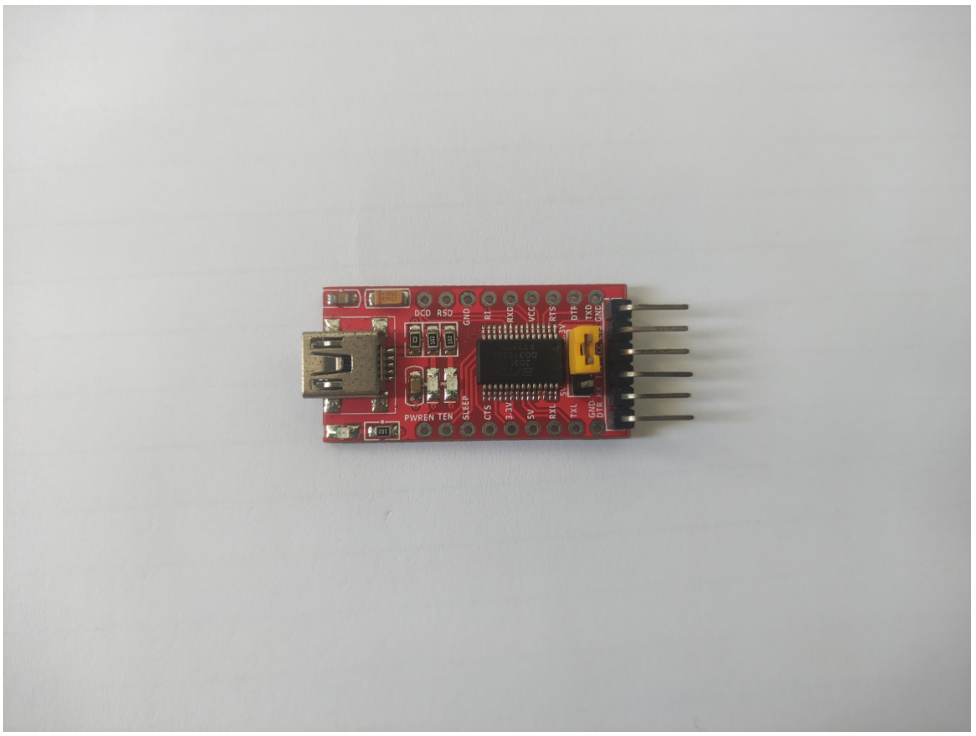
O módulo FTDI FT232RL (ver Figura 12) desempenha no projeto o papel de converter os dados de UART para USB, atuando como um elo de conexão entre os elementos da comunicação que utilizam diferentes interfaces.

Este dispositivo proporciona a conversão de sinal serial TTL para USB (MiniUSB, especificamente), através do uso do chip FT232R. Tal chip é fabricado pela empresa *Future Technology Devices International* - FTDI e apresenta como características principais, conforme International (2020):

- Protocolo USB genrenciado no chip;
- Taxas de transferência de dados de 300 *baud* a 3 M*baud* em níveis TTL;
- *Buffer* de recebimento de 128 *bytes* e *buffer* de transmissão de 256 *bytes*.

Ademais, faz-se importante abordar a lógica TTL, uma vez que esta é fundamental para funcionamento do conversor. Dito isso, TTL é uma família de circuitos digitais

Figura 12 – Módulo Conversor FTDI FT232RL.



Fonte: Elaborado pelo autor.

composto por transistores bipolares junção, diodos e resistores. Ela se relaciona com sistemas de comunicação ao implementar, segundo Tocci, Widmer e Moss (2011), níveis de tensão específicos para representar os estados lógicos, sendo o nível baixo indicado por valores de 0V a 0,8V e o alto indicado por valores de 2,0V a 5,0V.

3.3 ASCII

O Código Padrão Americano para o Intercâmbio de Informação (do inglês, *American Standard Code for Information Interchange* - ASCII) é um código binário para caracteres alfanuméricos que usa sete bits para codificar 128 caracteres (MANO; CILETTI, 2013).

Os sete bits do código são identificados como b_1 a b_7, onde b_7 representa o bit mais significativo. No entanto, a maioria dos computadores trabalha com oito bits como unidade básica, formando o byte. Portanto, os caracteres ASCII normalmente são armazenados em bytes, com o bit extra sendo utilizado para outros propósitos como extensão de conjunto de caracteres.

O ASCII geralmente é visto em forma de tabela, dessa forma, na Tabela 1 é apresentado um recorte específico da tabela ASCII, contendo apenas números, os caracteres especiais menos (`'-'`) e ponto (`'.'`); e os caracteres de controle, caractere nulo (NUL) e quebra de linha (LF). Essa seleção se justifica pelo fato de que apenas esses caracteres são

utilizados pela SCI durante a comunicação. Ademais, o bit extra será designado na tabela por conveniência como b_8.

Tabela 1 – Tabela ASCII dos caracteres utilizados no projeto.

Caractere	Código ASCII	Binário							
		b_8	b_7	b_6	b_5	b_4	b_3	b_2	b_1
NULL	00	0	0	0	0	0	0	0	0
LF	10	0	0	0	0	1	0	1	0
-	45	0	0	1	0	1	1	0	1
.	46	0	0	1	0	1	1	1	0
0	48	0	0	1	1	0	0	0	0
1	49	0	0	1	1	0	0	0	1
2	50	0	0	1	1	0	0	1	0
3	51	0	0	1	1	0	0	1	1
4	52	0	0	1	1	0	1	0	0
5	53	0	0	1	1	0	1	0	1
6	54	0	0	1	1	0	1	1	0
7	55	0	0	1	1	0	1	1	1
8	56	0	0	1	1	1	0	0	0
9	57	0	0	1	1	1	0	0	1

Fonte: Elaborado pelo autor.

Por meio da Tabela 1, é possível observar que se for necessário fazer o envio do caractere 5, por exemplo, o programa irá interpretá-lo como 00110101 para realizar a transmissão.

3.4 PyQt5 - Comprehensive Python Bindings for Qt v5

A interface gráfica do supervisório foi desenvolvida utilizando o PyQt5, uma ferramenta baseada em Qt. O Qt é um conjunto de bibliotecas C++ multiplataforma que fornece Interfaces de Programação de Aplicação de alto nível capazes de interagir com diversos aspectos de sistemas modernos para desktop e dispositivos móveis (COMPUTING, 2024).

O *Comprehensive Python Bindings for Qt v5*, por sua vez, é um *binding* que permite utilizar a linguagem de programação Python como alternativa ao C++ na criação de aplicações com o Qt v5 (versão 5) (COMPUTING, 2025).

Optou-se por utilizar o Qt devido sua riqueza de recursos e desempenho otimizado. Além disso, a escolha da programação em *Python* se deu pela maior flexibilidade e simplicidade que essa linguagem apresenta em detrimento ao C++.

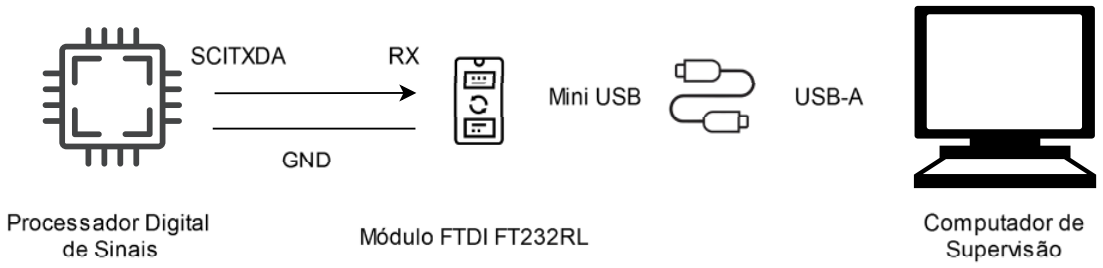
4 METODOLOGIA

O presente capítulo tem objetivo de descrever o *setup* do projeto, a rotina de envio dos dados efetuada pela DSP e o desenvolvimento do sistema supervisório utilizando o PyQt5.

4.1 Montagem

O início da implementação do projeto se dá pela montagem de seu *setup*. Dessa forma, a Figura 13 retrata como foram feitas as conexões dos dispositivos envolvidos.

Figura 13 – Montagem dos dispositivos do projeto.



Fonte: Elaborado pelo autor.

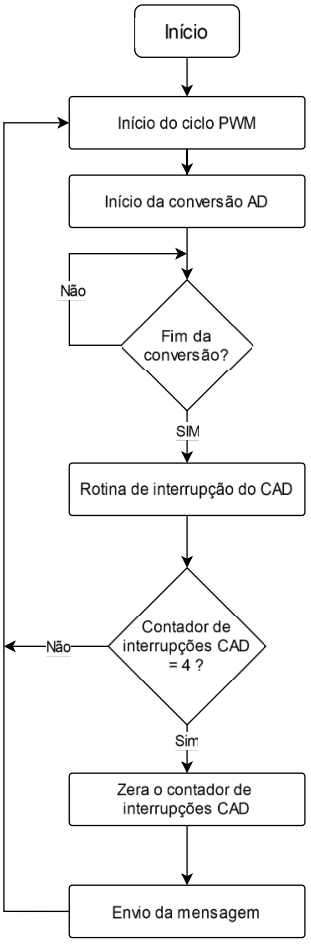
Para efetuar a transferência de dados, utilizou-se a porta SCITXDA do módulo SCI-A, correspondente ao pino 3 da placa DSP. Este pino foi conectado à entrada RX do módulo FT232RL. Além disso, foi estabelecido um aterramento comum, conectando o pino 20 da placa ao terminal GND do módulo. A transmissão de dados do FT232RL para o computador de supervisão foi realizada por meio de um cabo MiniUSB para USB-A.

4.2 Programação do DSP28335

O algorítimo desenvolvido integrou uma funcionalidade de envio de dados a um código-fonte base, escrito em linguagem C, utilizado no LAMP para realizar o controle de máquinas elétricas. Devido a políticas de privacidade do laboratório, o algoritmo completo não pôde ser disponibilizado. No entanto, as partes relacionadas à configuração da comunicação e um exemplo de envio de dados foram incluídas no Apêndice A.

Desse modo, o fluxograma que descreve o funcionamento da rotina está apresentado na Figura 14 e organiza-se a partir da interação entre uma interrupção gerada por periféricos e o envio de dados implementado no laço infinito da *main*.

Figura 14 – Fluxograma do algoritmo implementado no DSP.



Fonte: Elaborado pelo autor.

O código-fonte base tem como estrutura principal uma interrupção vinculada aos módulos PWM (*Pulse Width Modulation*) e ADC (*Analog-to-Digital Converter*). O módulo PWM opera a uma frequência de 10kHz e o disparo para início da conversão analógica-digital é realizado junto ao início de cada ciclo PWM. Quando as conversões são concluídas, o ADC dispara uma interrupção e é chamada a função onde está a rotina de controle do sistema, que processa dados convertidos e atualiza variáveis necessárias conforme o algoritmo de controle implementado.

Assim, a rotina de envio de dados utiliza um contador que registra o número de vezes em que a interrupção ocorre e é monitorado continuamente na *main*. Ao atingir 4 interrupções, o contador é reiniciado e os dados são enviados.

Como a parte de comunicação está inserida na *main*, sempre que a interrupção é disparada durante uma transmissão, ocorre uma troca de contexto que modifica o fluxo do programa. Nesse momento, a transmissão é temporariamente pausada, sendo retomada após o término da ISR. Contudo, é importante destacar que, caso a transmissão de um

dado já tenha sido iniciada pelo registrador de transmissão, o envio do caractere em curso será finalizado, porque a SCI opera independentemente do núcleo da CPU.

Com base nesses aspectos, o tempo disponível para execução da rotina de envio corresponde a 100μs, sendo este o intervalo entre as interrupções, subtraído do tempo de execução da ISR. Durante os testes, constatou-se que a interrupção demora em média de 60μs, resultando em 40μs disponíveis para execução das demais tarefas. Como o tempo máximo para execução da rotina de envio é de aproximadamente 193,61μs (conforme será mostrado na subseção 4.2.4), ao dividi-lo por 40μS, obtém-se o valor de aproximadamente 4,84. Esse resultado indica que são necessárias quatro interrupções para realizar todo processo de comunicação.

Outra questão envolvendo a quantidade de interrupções apresentada é a amostragem associada à reconstrução do sinal que chega ao receptor. Considerando que geração dos dados de interesse ocorre a cada 10kHz e que um dado é transmitido a cada quatro interrupções, tem-se uma nova amostragem de 2500Hz. Como informado na subseção 1.2.1, um dos requisitos do projeto é que o sistema supervisório seja capaz de apresentar corretamente sinais com frequência mínima de 60 Hz.

Portanto, é preciso determinar se o dado que chega ao receptor ainda representa corretamente o sinal original. Segundo Haykin (2013), um sinal é completamente recuperado quando sua frequência de amostragem é de pelo menos, o dobro da frequência máxima presente no sinal original, sendo esta chamada de taxa de *Nyquist*. Assim, como 2500 Hz é um valor superior a 120 Hz, esse é atendido.

Neste ponto se faz importante mencionar que devido limitações relacionadas ao *driver* encontrado para o Conversor FTDI FT232RL, o máximo *baud rate* aplicado a comunicação foi 921600 bits por segundo (bps). Implicando no tempo de envio mencionando e, consequentemente, na quantidade de interrupções.

4.2.1 Configuração da Interface de Comunicação Serial

A configuração deste periférico, bem como a implementação das funções responsáveis pelo envio de dados, foi realizada por meio da manipulação direta dos registradores do dispositivo e teve como base um código exemplo chamado **Example_2833xSci_Echoback.c** presente na documentação do DSP.

Dessa maneira, dentre os modos de comunicação disponíveis para interface, o adotado para este projeto foi o modo multiprocessador de linha ociosa, tendo em vista que ele não adiciona um bit extra ao *frame* e é compatível com a comunicação do tipo RS232. Além disso, se fez necessário habilitar o pino GPIO (*General-Purpose Input/Output*, ou Entrada/Saída de Propósito Geral) 28, vinculado ao módulo SCI-A, para atuar como transmissor (SCITXDA).

Em se tratando das configurações do *frame*, tem-se que foi empregado o formato 8-N-1, uma vez que ele permite a transmissão de 8 bits de dados por *frame*, sendo suficiente para representar, conforme a tabela ASCII, todos os caracteres necessários a representação dos dados. Considerando que o tempo de envio da mensagem é um fator crucial, optou-se por não empregar o bit de paridade e mais de um bit de parada, de modo a evitar sobrecarga na comunicação.

Outra etapa da configuração da comunicação é a definição do *baud rate*. No entanto, antes de se abordar diretamente esse valor, é preciso mencionar que ele depende de uma constante chamada LSPCLK ou *Clock* de baixa frequência. Essa constante, é um sinal de *clock* utilizado pelo DSP28335 para operar periféricos de baixa velocidade, como o SCI, SPI e I2C, e é calculada por padrão como um quarto da frequência de funcionamento da CPU. Desse modo, sabendo que a frequência de CPU utilizada no projeto é 150MHz, temos LSPCLK igual a 37,5MHz.

O *baud rate* é ajustado por meio de dois registradores de oito bits: **SCIHBAUD** (onde fica o *byte* mais significativo) e **SCILBAUD** (onde fica o *byte* menos significativo), que são concatenados para formar um *baud* de até 16 bits, denominado BRR. Assim, o BRR é calculado de acordo com a Equação 4.1:

$$BRR = \frac{LSPCLK}{\text{baud rate SCI} \cdot 8} - 1 = \frac{37500000}{921600 \cdot 8} - 1 \Longrightarrow BRR \approx 4,08 \tag{4.1}$$

Logo, o valor de BRR utilizado para alcançar um *baud rate* de 921600 foi 4, considerando que os registradores operam exclusivamente com valores inteiros. Ademais, foi necessário apenas o uso do registrador SCILBAUD, dado que o resultado obtido está contido em 8 bits, não exigindo registros adicionais.

Por fim, fez-se a configuração das filas FIFO do módulo SCI. Conforme Axelson (2007), FIFO é um espaço de armazenamento cujo conteúdo é lido na mesma ordem em que foi armazenado. Desse modo, foi habilitado o uso da fila de transmissão e desativada a geração de interrupções baseadas em seu nível de preenchimento.

4.2.2 Função para conversão de tipo de dados

As variáveis presentes em códigos aplicados a processadores digitais de sinais geralmente utilizam tipos de dados numéricos, como *int*, *long*, *float* e *double*. No entanto, esses tipos de dados não podem ser enviados diretamente pela SCI, pois são representados internamente por estruturas binárias específicas, cujo tamanho e codificação variam de acordo com o tipo.

Dada a complexidade de criar uma função que lide de maneira eficiente e precisa com todos esses tipos de dados, optou-se por trabalhar especificamente com o formato

float. Esse formato tem capacidade de representar números com parte inteira e decimal, com precisão suficiente para a maioria dos casos práticos, além de exigir menos memória e processamento em comparação com o *double*.

Assim, foi desenvolvida uma função chamada **numeroParaString()** que converte valores no formato *float* para uma sequência de caracteres, armazenada em um vetor de *char* (ponteiro para *char*). Essa conversão transforma os dados em uma sequência de códigos numéricos compatíveis com a tabela ASCII, permitindo que sejam transmitidos de maneira padronizada.

Para armazenar a sequência de caracteres que representa o número convertido, o vetor de *char* utilizado possui tamanho oito. Esse tamanho foi escolhido para acomodar até sete dígitos da representação numérica, garantindo um equilíbrio entre o espaço ocupado e a capacidade de representar os números mais comuns com precisão adequada. O caractere adicional é reservado para o terminador nulo (\0), utilizado em C para indicar o final da *string*.

Dito isso, a função **numeroParaString()** começa verificando se o número é negativo, adicionando o sinal de menos à primeira posição do vetor de char em caso afirmativo. Em seguida, processa a parte inteira e a parte decimal do número separadamente. Para a parte inteira, os dígitos são extraídos e convertidos para caracteres, sendo armazenados um a um no vetor de char. Caso haja uma parte decimal, ela é processada até quatro dígitos após o ponto decimal, convertendo cada dígito para o formato de caractere. Por fim, a função adiciona um caractere nulo (\0) ao final da string para indicar o término.

4.2.3 Funções para envio de dados

O envio de dados no módulo SCI é realizado pela integração entre as funções **scia_xmit()** e **scia_msg()**, utilizando o registrador SCITXBUF como ponto central. O SCITXBUF armazena temporariamente os dados a serem transmitidos, transferindo-os para a FIFO de transmissão, que organiza o envio sequencial pelo barramento serial.

A função **scia_xmit()** realiza a transmissão de um único caractere, garantindo que a FIFO de transmissão esteja disponível antes de carregar o dado no SCITXBUF. Já a função **scia_msg()** permite o envio de uma mensagem completa, percorrendo caractere por caractere do vetor de caracteres recebido como parâmetro e chamando **scia_xmit()** para transmitir cada dado de forma individual e ordenada.

4.2.4 Análises do tempo de envio

O tempo gasto pela rotina de envio para realizar a transmissão dos dados é calculado a partir do tempo necessário para transmitir um *frame*. No formato 8-N-1, com um *baud rate* de 921600 bps, o tempo de transmissão de um *frame* (T_{frame}) pode ser obtido pela

razão entre o número total de bits no *frame* (10 bits) e o *baud rate*, como mostrado na Equação 4.2:

$$T_{frame} = \frac{num_bits}{baud_rate} = \frac{10}{921600} = 0.00001085069s \Longrightarrow T_{frame} \approx 10,85\mu s \tag{4.2}$$

A rotina foi projetada para enviar números com até 7 dígitos e para delimitar o término de cada número, foi utilizado o caractere especial de quebra de linha (\n). Essa abordagem resulta em um total de 8 *frames* transmitidos por número, considerando os 7 dígitos e o caractere delimitador. O uso do caractere \n é essencial para garantir que o receptor interprete corretamente a mensagem, identificando de forma clara onde termina cada número.

A transmissão inclui o envio de dois números consecutivos, o que requer a transmissão de 16 *frames* no total. Além do tempo necessário para transmitir esses *frames*, deve-se incluir o tempo de execução da função de conversão de tipos de dados **numeroParaString()**, a qual consome $10\mu s$. Esse tempo extra foi de relevante para o cálculo do tempo total de execução da rotina, pois é preciso considerar tanto os aspectos de transmissão quanto os de processamento no desempenho geral do sistema. Portanto, o tempo total do envio de dados T_{envio} é: $(10,85\mu * 16) + 20 \approx 193,61\mu s$.

Além dos cálculos realizados para determinar os tempos envolvidos na transmissão de dados, foram capturadas imagens da tela de um osciloscópio, onde é possível observar e validar os tempos descritos matematicamente. Dessa forma, a Figura 15 apresenta os sinais seriais enviados durante a transmissão do caractere 5. Tais sinais foram obtidos conectando-se a ponteiro do osciloscópio ao pino referente a porta SCITXDA.

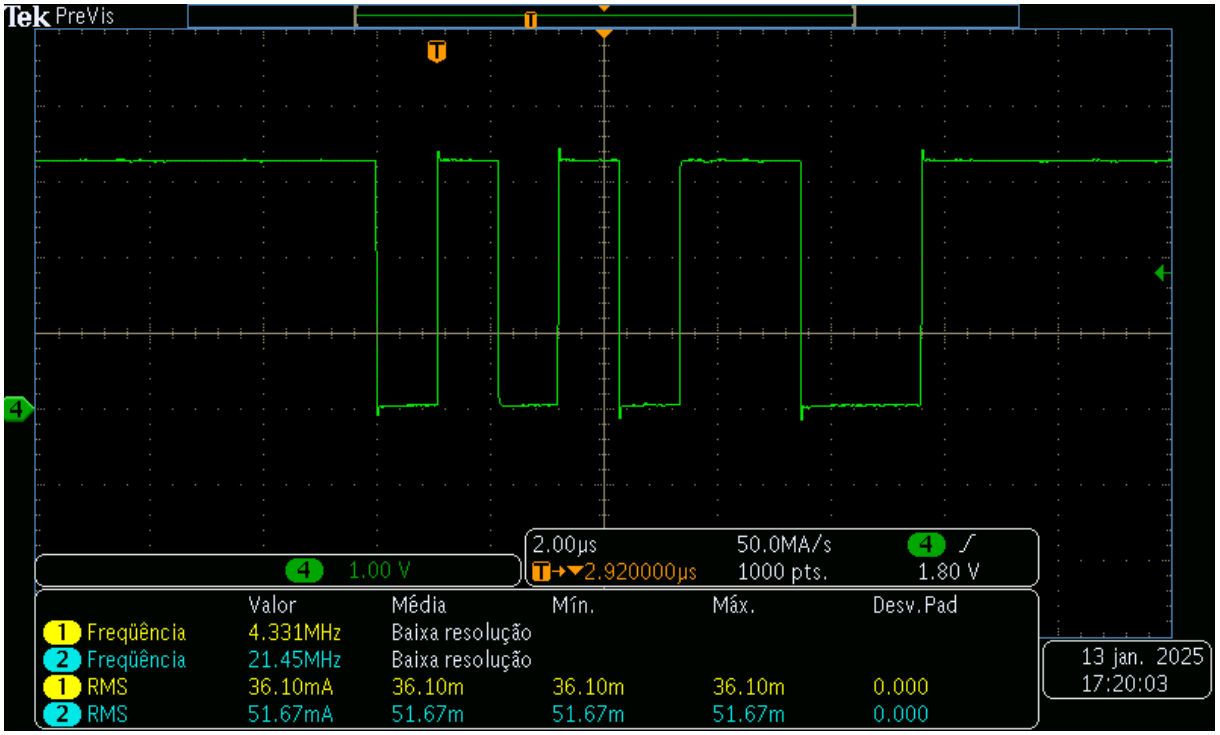
A partir da Figura 15, observa-se que o tempo medido experimentalmente para a transmissão do *frame* analisado se aproxima do tempo teórico. Além disso, confirma-se a teoria previamente discutida na subseção 2.2.1 acerca da organização e disposição dos bits no processo de comunicação UART, onde têm-se da esquerda para direta: o *bit* de início em estado lógico baixo, os *bits* de dados 10101100, conforme previsto pela Tabela ASCII e o *bit* de parada, que está implícito, pois tem o mesmo estado da linha de transmissão.

A Figura 16 contém os sinais monitorados durante a execução do algoritmo implementado e descrito nesta seção. Nela, são exibidos sinais distintos correspondentes três processos: a execução da rotina de interrupção, em rosa; a transmissão de dois dados via SCI, em verde; e a execução completa da rotina de envio de dados, em azul.

Para realizar a visualização desses sinais, além da conexão da ponteira descrita para Figura 15, as demais conexões foram feitas a pinos digitais configurados via software para assumir o estado lógico 1 no início de cada evento e retornam ao estado lógico 0 ao final.

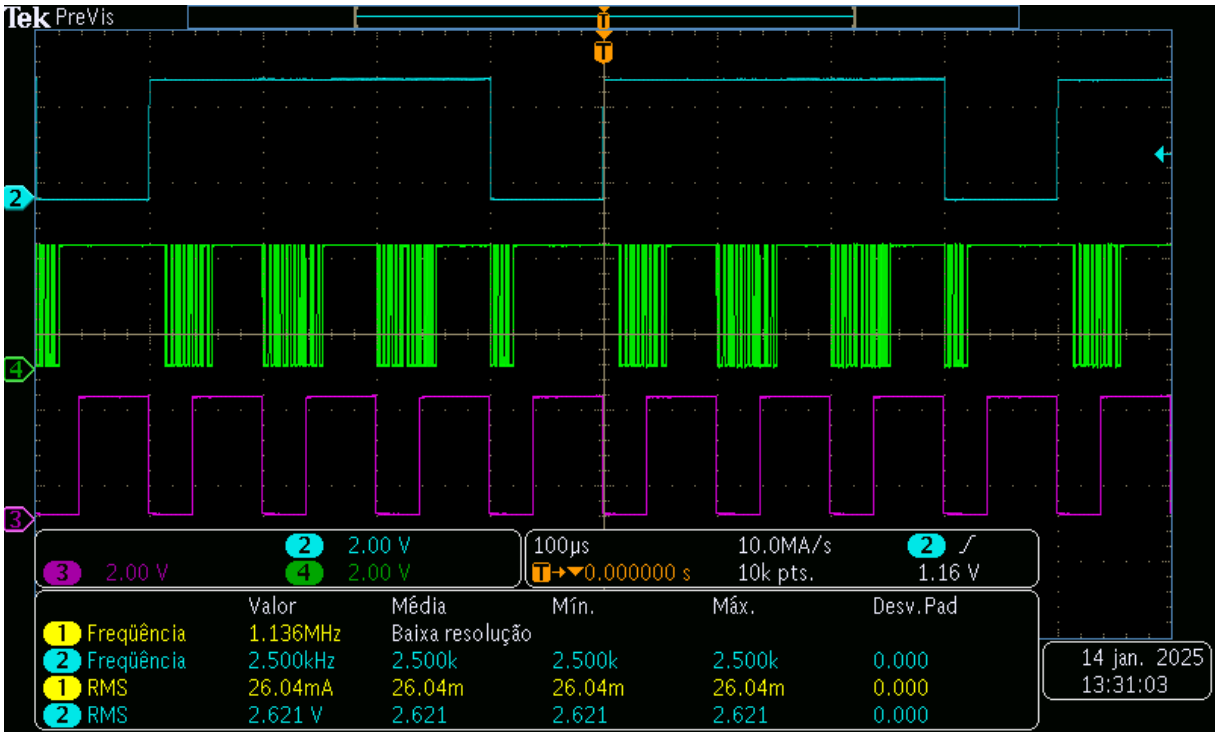
Dessa maneira, o primeiro pulso azul, da esquerda para a direita, indica o início

Figura 15 – *Frame* do caractere 5.



Fonte: Elaborado pelo autor.

Figura 16 – *Frame* do caractere 5.



Fonte: Elaborado pelo autor.

do envio de um *frame*. Esse evento pode ser identificado como tal, pois é precedido pela queda de um pulso rosa, que indica o término da execução da rotina de interrupção, e

seguido por uma sequência de sinais alternados, os quais correspondem aos pulsos que compõem o *frame*.

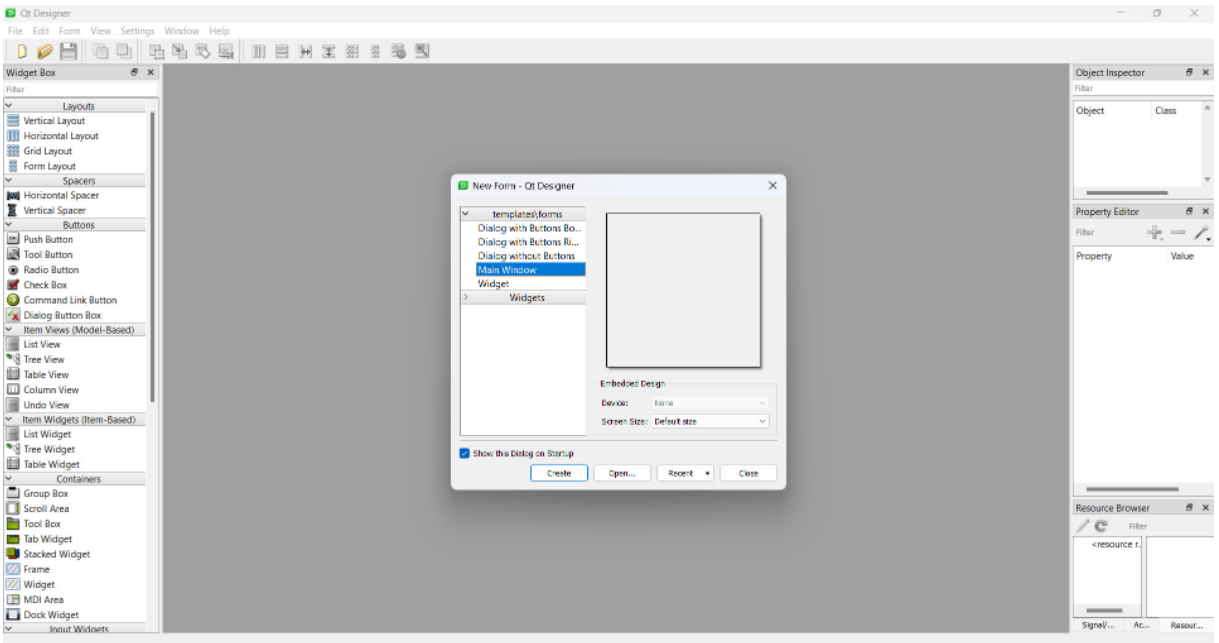
Mais adiante, o intervalo presente entre os sinais do *frame* representa a troca de contexto para execução da ISR. Esse processo se repete até que todos os dados tenham sido enviados, fazendo com que o pulso azul volte para 0. Após isso, tem-se a quarta interrupção e uma nova execução da rotina de envio.

4.3 Desenvolvimento do supervisório

O desenvolvimento do supervisório foi dividido em duas etapas principais: a construção da interface gráfica do usuário (do inglês, *Graphical User Interface* - GUI) e a programação de sua lógica de funcionamento.

Na primeira etapa, a GUI foi projetada utilizando o Qt *Designer*, uma ferramenta visual e interativa que simplifica a criação de interfaces gráficas por permitir o desenvolvimento sem a necessidade de escrever manualmente o código de *layout*. Através do Qt *Designer*, foi possível estruturar a aplicação, organizando *widgets*, além de definir a disposição desses componentes na janela principal. "No Qt, *widget* é o nome dado a um componente da interface do usuário com o qual o usuário pode interagir" (FITZPATRICK, 2020, p. 32). A Figura 17 mostra a tela inicia do Qt *Designer*.

Figura 17 – Tela inicial do Qt *Designer*.



Fonte: Elaborado pelo autor.

A segunda etapa consistiu na programação da lógica de funcionamento da GUI desenvolvida no ambiente programção *Visual Studio Code* (VS Code). O VS Code é um

editor de código-fonte multiplataforma, gratuito e de código aberto, desenvolvido pela Microsoft, que oferece depuração integrada, configurações personalizáveis e uma ampla biblioteca de extensões para diversas linguagens e *frameworks*.

Nessa fase, os elementos visuais foram conectados às funcionalidades do sistema, definindo como a aplicação responde às interações do usuário. Essas conexões são feitas por meio da lógica de sinais e *slots*.

Sinais são notificações emitidas por *widgets* para indicar a ocorrência de eventos, como o clique de um botão. Eles podem ser desencadeados por ações do usuário ou gerados automaticamente pelo sistema. Além de indicar o evento, os sinais são capazes de transmitir dados para contextualizar sua ocorrência. Já os *slots* são os receptores dos sinais. Em python qualquer função ou método pode atuar como um *slot*, desde que um sinal esteja associado a ele. Nos casos em que o sinal envia dados, o *slot* os recebe automaticamente (FITZPATRICK, 2020).

4.3.1 Interface do Supervisório

A interface do supervisório foi construída visando atender os seguintes requisitos do usuário:

- Seleção da porta serial para comunicação.
- Configuração da quantidade e do tipo de curvas a serem exibidas.

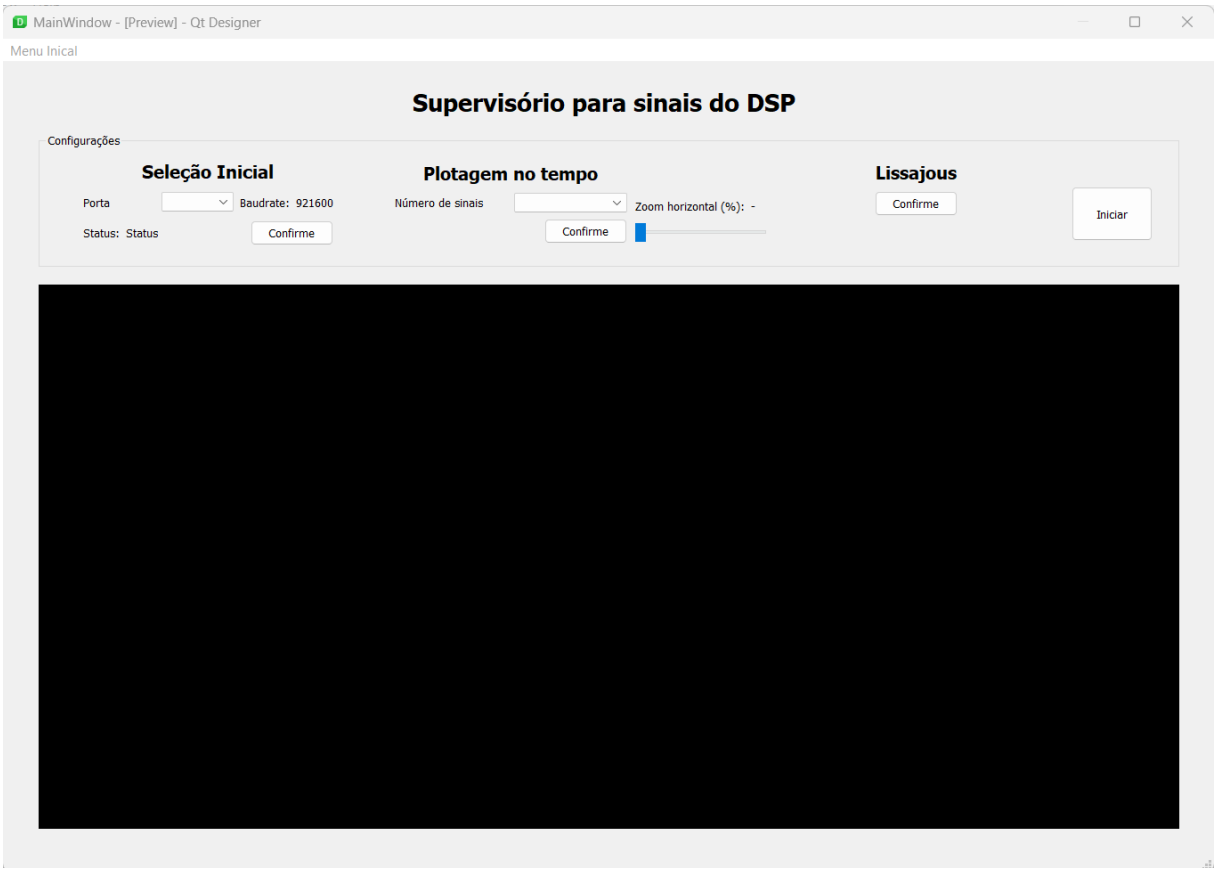
Desse modo, a figura Figura 18 apresenta a tela inicial do interface, destacando os elementos que permitem realizar essas configurações de forma prática e intuitiva.

Para o correto funcionamento do supervisório, é necessário seguir um passo a passo de configuração previamente definido. O processo inicia-se na aba "Configurações", na seção "Seleção Inicial". Ao executar o supervisório com um ou mais dispositivos conectados à porta serial, o usuário pode, por meio de uma caixa de combinação, selecionar a porta pela qual será estabelecida a comunicação. Após realizar a escolha, é necessário pressionar o botão "Confirme" para concluir esta etapa da configuração.

Em seguida, o usuário deve decidir se os sinais serão exibidas em função do tempo ou no formato Lissajous. Em ambos os casos, o quadro preto é destinado à exibição das curvas.

A configuração da exibição dos sinais em função do tempo é feita na seção "Plotagem no tempo". Nesse caso, o usuário pode optar por visualizar um ou dois sinais e, ao pressionar o botão "Confirme", um ou dois eixos de coordenadas são automaticamente inseridos no quadro preto, conforme a opção selecionada. A Subfigure 19 mostra a interface configurada para apresentar os dados nos casos descritos.

Figura 18 – Tela inicial do supervisório.

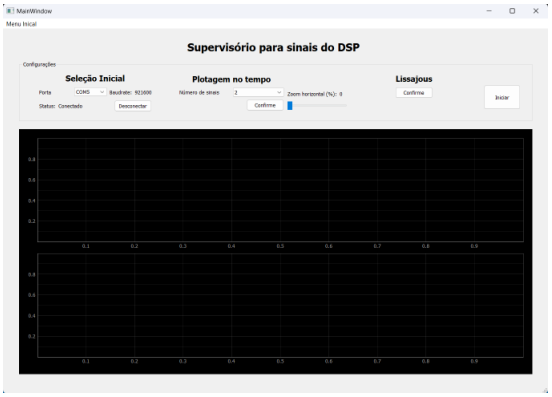


Fonte: Elaborado pelo autor.

Figura 19 – Interface configura para plotagem de sinais em função do tempo

(a) Quantidade de sinais igual a 1

(b) Quantidade de sinais igual a 2



Fonte: Elaborado pelo autor.

A exibição dos sinais no formato Lissajous, é feita ao se pressionar o botão "Confirme" na seção "Lissajous". Nesse caso, um eixo cartesiano será inserido no quadro preto similar ao que foi mostrado na Subfigure 19 (a).

Após configurar a porta serial e definir o modo de visualização e a quantidade de

sinais a serem traçados, o usuário deve pressionar o botão "Iniciar". Esse comando dispara o evento de leitura da porta serial e a consequente exibição dos sinais na interface.

Com o sistema em funcionamento, no caso em que foi escolhido ver sinais em função do tempo, o usuário pode ainda ajustar o zoom horizontal das curvas utilizando o *slider* localizado ao lado do botão "Confirme" nesta mesma seção. Essa funcionalidade permite ampliar ou reduzir a escala temporal das curvas exibidas, proporcionando uma análise mais detalhada ou uma visão geral dos dados.

Ademias, a criação da interface gerou um arquivo no formato .ui, que contém a descrição da estrutura visual da aplicação e está no Apêndice B. Esse arquivo .ui é posteriormente chamado e utilizado na programação da aplicação, permitindo a integração da interface gráfica com o código funcional de maneira eficiente.

4.3.2 Estrutura da lógica da aplicação

A aplicação (ver Apêndice C) foi implementada com base em duas classes principais, organizadas de forma a separar a interface gráfica da aquisição de dados:

- **class MyGUI(QMainWindow)**: é responsável pela interface gráfica e pela interação com o usuário, implementando sinais e slots que gerenciam os eventos.
- **class DataAcquisitionThread(QThread)**: gerência a leitura dos dados e os disponibiliza de forma síncrona para a exibição em tempo real.

A divisão foi adotada porque a leitura da porta serial ocorre de forma contínua e assíncrona. Caso fosse realizada no mesmo contexto da interface gráfica, a execução simultânea das tarefas poderia gerar atrasos ou travamentos. Para mitigar esses problemas, optou-se por isolar a aquisição de dados, adotando uma abordagem que possibilite a execução independente de múltiplas tarefas.

Assim, a abordagem escolhida foi o uso de *threads*, que são unidades básicas de execução da CPU. No contexto do PyQt5, a classe QThread é utilizada para implementar *threads* de maneira eficiente em aplicações gráficas, evitando que tarefas intensivas ou bloqueantes, como a leitura contínua de dados da porta serial, interrompam ou comprometam a fluidez da interface gráfica.

Para permitir que os dados lidos pela porta serial sejam transferidos de uma *thread* para outra foram utilizados *buffers* compartilhados e a execução da conexão entre as duas *threads* é iniciada quando o botão "Iniciar" da interface gráfica é pressionado. Nesse momento, a lógica presente na classe MyGUI configura a aquisição de dados, ajusta os parâmetros dos gráficos, cria uma instância da classe **DataAcquisitionThread** e aciona um temporizador.

O temporizador é configurado para executar uma rotina responsável por atualizar em intervalos constantes de 1 segundo os gráficos com as informações presentes nos *buffers* compartilhados. A justificativa que embasa essa escolha de tempo é que com ele a quantidade de ciclos exibidos corresponde exatamente à frequência do sinal mostrado, proporcionando uma visualização alinhada ao comportamento do dado.

Tendo sido instanciada, a *thread* de aquisição é iniciada. A leitura dos dados recebidos pela porta serial é gerenciada de maneira a lidar com diferentes configurações de sinais, podendo ser um único dado ou pares de dados. Desse modo, os dados são lidos caractere por caractere por meio do método read(1) e armazenados em um *buffer* temporário. Quando o caractere \n é detectado, o conteúdo do *buffer* é tratado como um dado completo, que é então convertido para um número em ponto flutuante. Caso a conversão seja bem-sucedida, o dado é processado; caso contrário, o *buffer* é descartado, e o sistema aguarda novos dados, assegurando que apenas informações válidas sejam utilizadas

No caso em que apenas um dado é lido por ciclo de aquisição, o valor convertido é armazenado em um *buffer* local denominado *buffer_y1*. Paralelamente, o marcador de tempo correspondente é calculado e armazenado no vetor *buffer_x*. Esse vetor é preenchido de maneira a manter correspondência temporal com os dados da coordenada y. A base temporal é gerada a partir do período de amostragem $T = 0,0004$, correspondente ao intervalo de tempo entre as amostras do sinal recebido. Inicialmente, o valor de x é definido como zero e, a cada dado recebido, é incrementado pelo valor de T, garantindo que os valores de x reflitam o intervalo de tempo entre as amostras.

Quando dois sinais são lidos simultaneamente, o programa adota uma estratégia de pareamento. O primeiro valor lido é temporariamente armazenado na variável *pending_pair*. O segundo valor, ao ser recebido, é associado ao primeiro, formando um par. Ambos os valores são então armazenados nos *buffers*, *buffer_y1* e *buffer_y2*, enquanto os marcadores temporais correspondentes são adicionados ao vetor *buffer_x*. Caso a sequência de leitura dos dados não seja respeitada, o pareamento não é realizado, e o dado é descartado, reforçando a consistência da aquisição.

Independentemente do cenário, os *buffers* locais são configurados para armazenar até 200 amostras. Quando esse limite é alcançado, os dados são transferidos para estruturas de armazenamento compartilhadas (*eixo_x*, *eixo_y1*, *eixo_y2*). Durante essa transferência, utiliza-se um mecanismo de exclusão mútua (*mutex*) para evitar condições de corrida, assegurando que a interface gráfica possa acessar os dados de forma segura. Após a transferência, os *buffers* locais são esvaziados e preparados para o processamento das próximas amostras.

Na atualização dos gráficos, o temporizador dispara uma rotina que acessa os *buffers* compartilhados, bloqueando temporariamente outras operações que os envolvam. Assim,

os gráficos são atualizados a depender da quantidade de sinais e formato predefinidos. Se o tipo de exibição for no tempo, as informações são inseridos no gráfico como (x,y), caso seja Lissajous os dois dados referentes ao eixo y são exibidos no formato (y,y).

A atualização do espaço horizontal da curva é realizada em tempo real conforme o ajuste do *slider*, permitindo controlar os limites do eixo xx e ajustar o intervalo exibido no gráfico. Esse controle atua como um zoom horizontal para os gráficos plotados em função do tempo. Como os dados são enviados a cada 0,0004 segundos e os gráficos atualizados a cada 1 segundo, o *buffer* compartilhado contém, em média, 2500 valores. O ajuste do *slider* define a fração de valores do *buffer* a ser utilizada na plotagem. Por exemplo, ao configurar o *slider* para o valor 10, 5% dos valores iniciais e finais do vetor são descartados, resultando na exibição de 2250 valores no gráfico. Isso facilita a visualização detalhada de regiões específicas do sinal analisado.

5 RESULTADOS

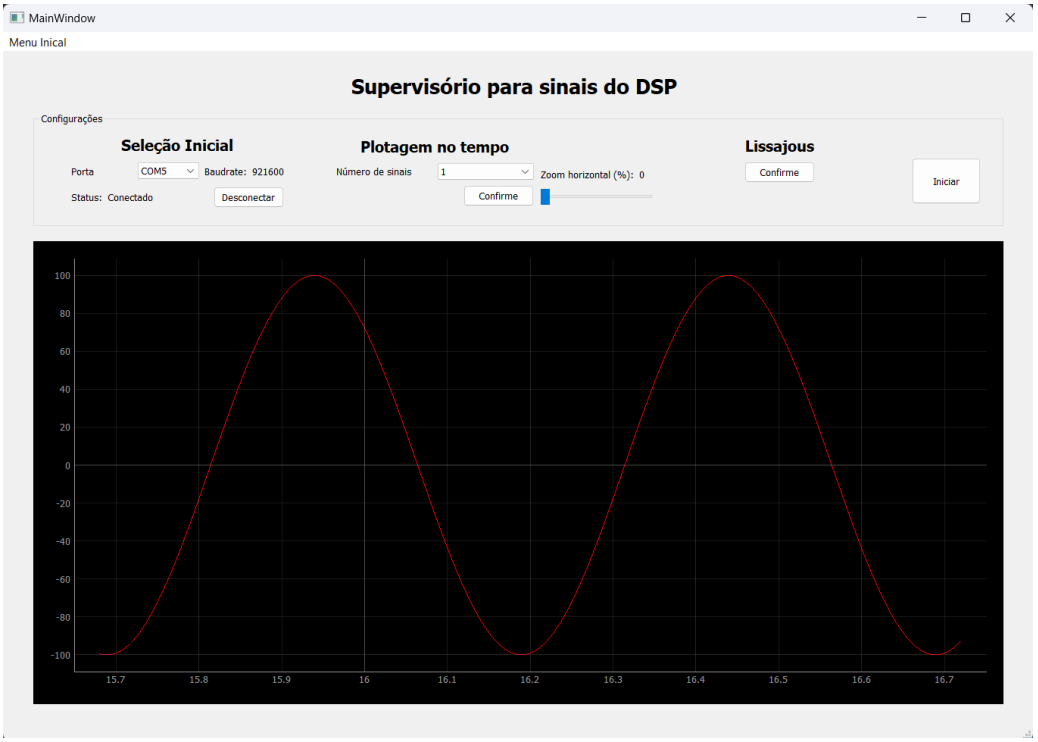
Foram realizados dois conjuntos de testes para validar a integração entre a rotina de envio de dados do DSP28335 e a exibição desses dados no sistema supervisório. No primeiro conjunto de testes, os sinais exibidos nos gráficos foram gerados internamente pelo DSP, com o objetivo de avaliar a funcionalidade básica do sistema de supervisão. No segundo conjunto de testes, o supervisório foi utilizado para monitorar sinais gerados durante o acionamento de um motor presente nas instalações do LAMP, permitindo a análise do desempenho em condições de aplicação real.

5.1 Testes simulados

Para os testes a seguir foi implementada na rotina de interrupção a geração de duas senoides com 100 de amplitude. Já no computador de supervisão, foi seguido o passo a passo de configuração descrito na subseção 4.3.1.

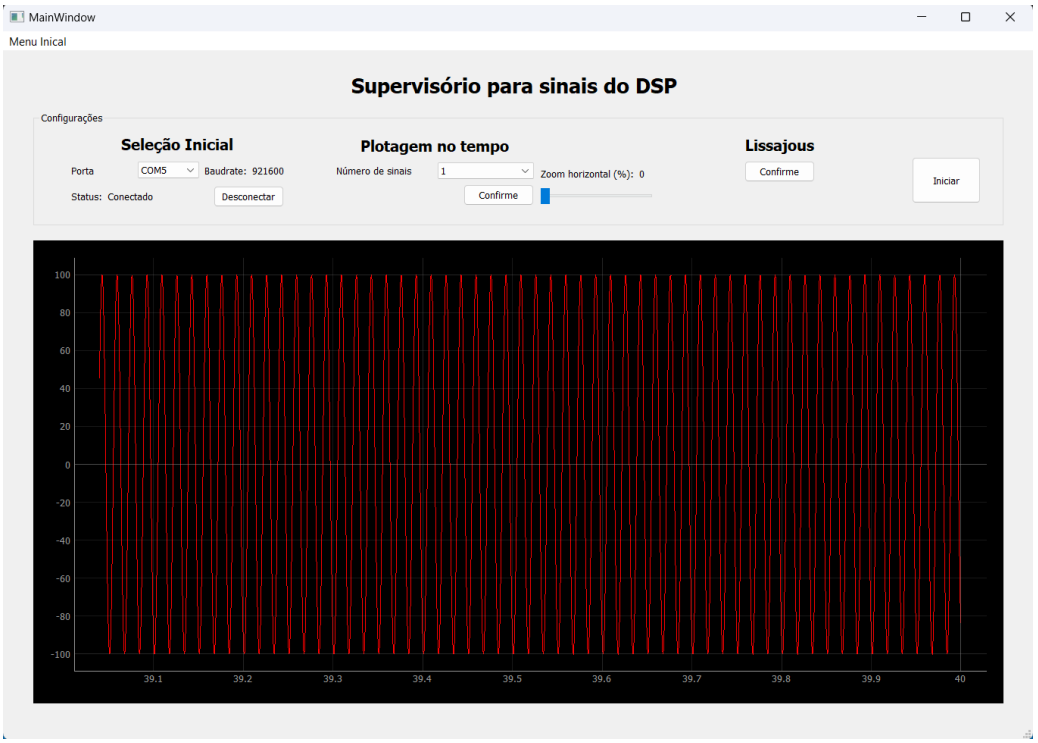
Inicialmente, foi feito o envio de apenas uma das senoides. A Figura 20 e a Figura 21 ilustram o funcionamento do supervisório com curvas de 2 Hz e 60 Hz, respectivamente, sendo representadas graficamente em função do tempo.

Figura 20 – Exibição no tempo de um sinal de 2 Hz.



Fonte: Elaborado pelo autor.

Figura 21 – Exibição no tempo de um sinal de 60 Hz.

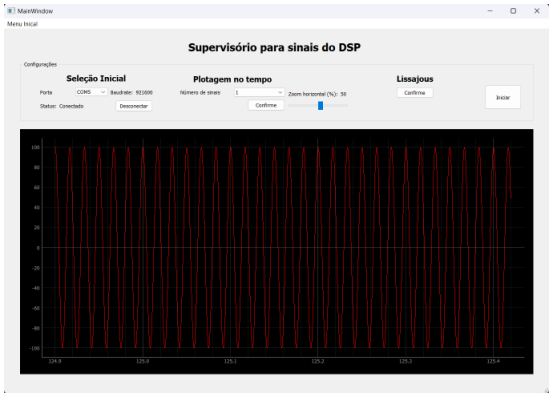


Fonte: Elaborado pelo autor.

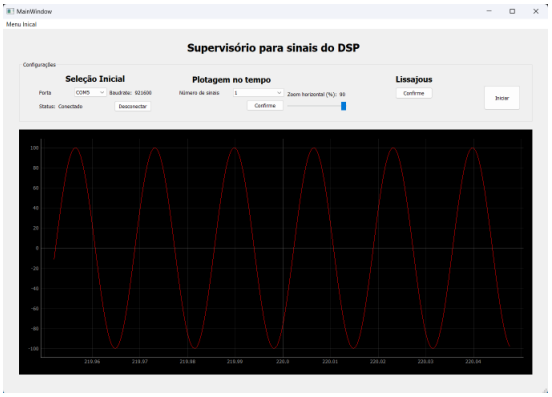
Ao observar a Figura 21, é possível notar que o sinal exposto apresenta difícil visualização, devido sua frequência ser relativamente alta para o intervalo de tempo amostrado na tela. Assim, a Subfigure 22 ilustra dois casos de uso do *zoom* horizontal.

Figura 22 – Exibição no tempo de um sinal de 60Hz com zoom horizontal aplicado

(a) *Zoom* horizontal de 50%



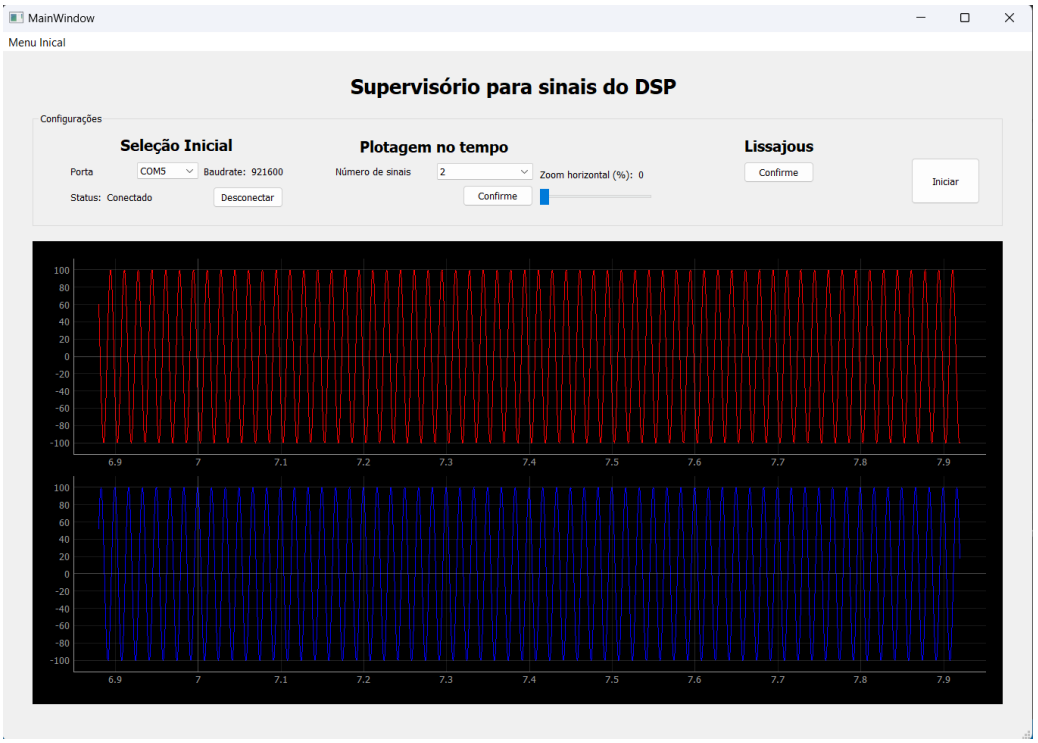
(b) *Zoom* horizontal de 90%



Fonte: Elaborado pelo autor.

Em seguida, alterou-se o código do DSP para realizar o envio de duas senoides. Desse modo, a Figura 23 fornece uma visualização do supervisório com dois sinais de 60 Hz em função do tempo, enquanto a Subfigure 24 apresenta a mesma visualização com o zoom aplicado.

Figura 23 – Exibição no tempo de dois sinais de 60 Hz defasados em 120°.

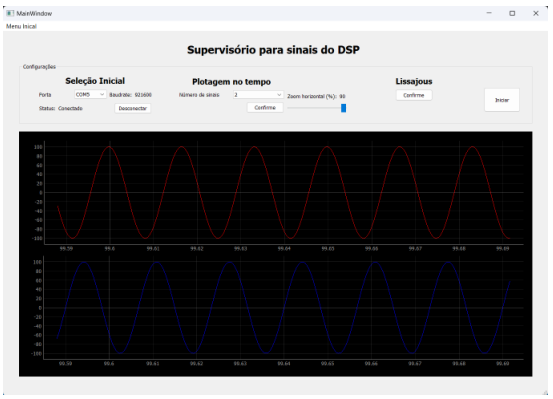


Fonte: Elaborado pelo autor.

Figura 24 – Exibição no tempo de dois sinais de 60Hz defasados em 120° com zoom horizontal aplicado.

(a) *Zoom* horizontal de 50%

(b) *Zoom* horizontal de 90%



Fonte: Elaborado pelo autor.

Ainda com o DSP enviando dados referentes a duas senoides, é possível utilizar a opção de exibição no formato Lissajous, ilustrado na Figura 25.

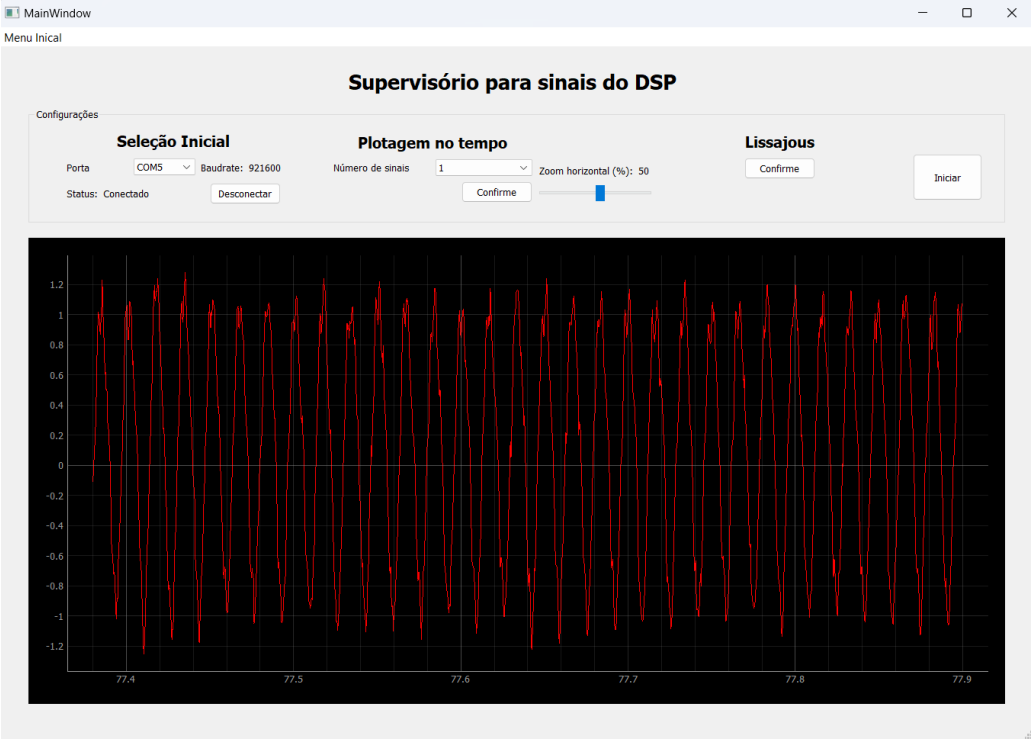
Figura 25 – Exibição no formato de Lissajous..

Fonte: Elaborado pelo autor.

5.2 Testes com motor em Operação

Neste conjunto de testes, as variáveis monitoradas foram correntes que percorrem os enrolamentos de um motor enquanto sua velocidade era de 1800 Rotações por Minuto (RPM). Inicialmente, o código do DSP28335 e o supervisório foram configurados para, respectivamente, enviar e exibir apenas um dado de corrente. Desse modo a Figura 26 ilustra o funcionamento do supervisório para o caso descrito.

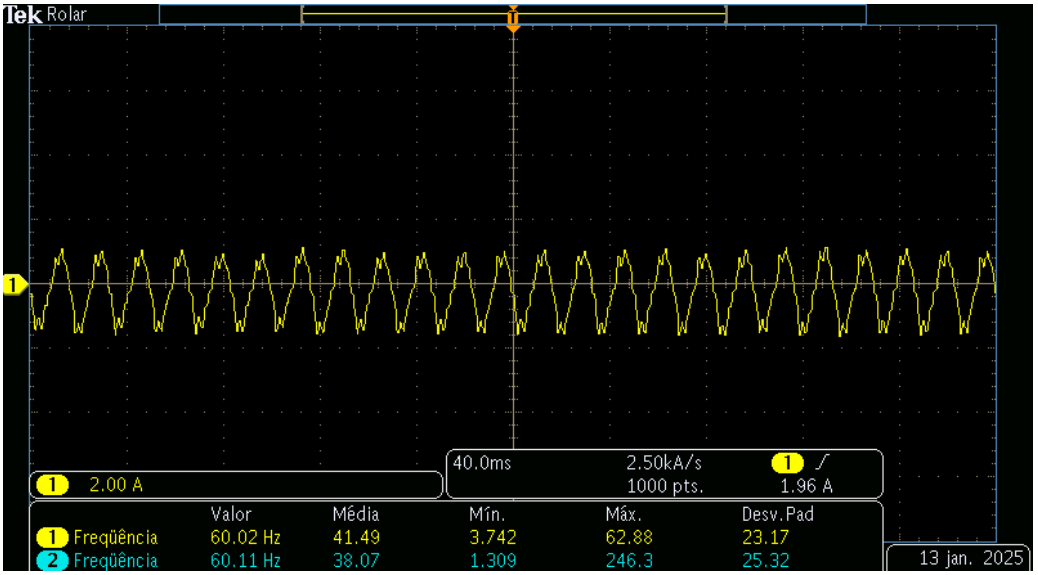
Figura 26 – Exibição no tempo de um sinal de corrente.



Fonte: Elaborado pelo autor.

Afim de estabelecer um nível de comparação e observar se os dados exibidos pelo supervisório estavam corretos, foram feitas capturas de tela de um osciloscópio que monitorava a mesma corrente do supervisório. Desse modo, a Figura 27 retrata a leitura de corrente feita pelo osciloscópio.

Figura 27 – Sinal de corrente medido pelo osciloscópio.



Fonte: Elaborado pelo autor.

Ao observar o sinal presente na Figura 27, é possível constatar que sua amplitude é de aproximadamente 2 Amperes (A) e que sua frequência é 60 Hz. Já no sinal presente na Figura 26, a amplitude oscila, mas fica numa faixa próxima dos 2 A e há aproximadamente 6 períodos do sinal dentro de um intervalo de 100 milissegundos, o que resulta em uma frequência de 60 Hz.

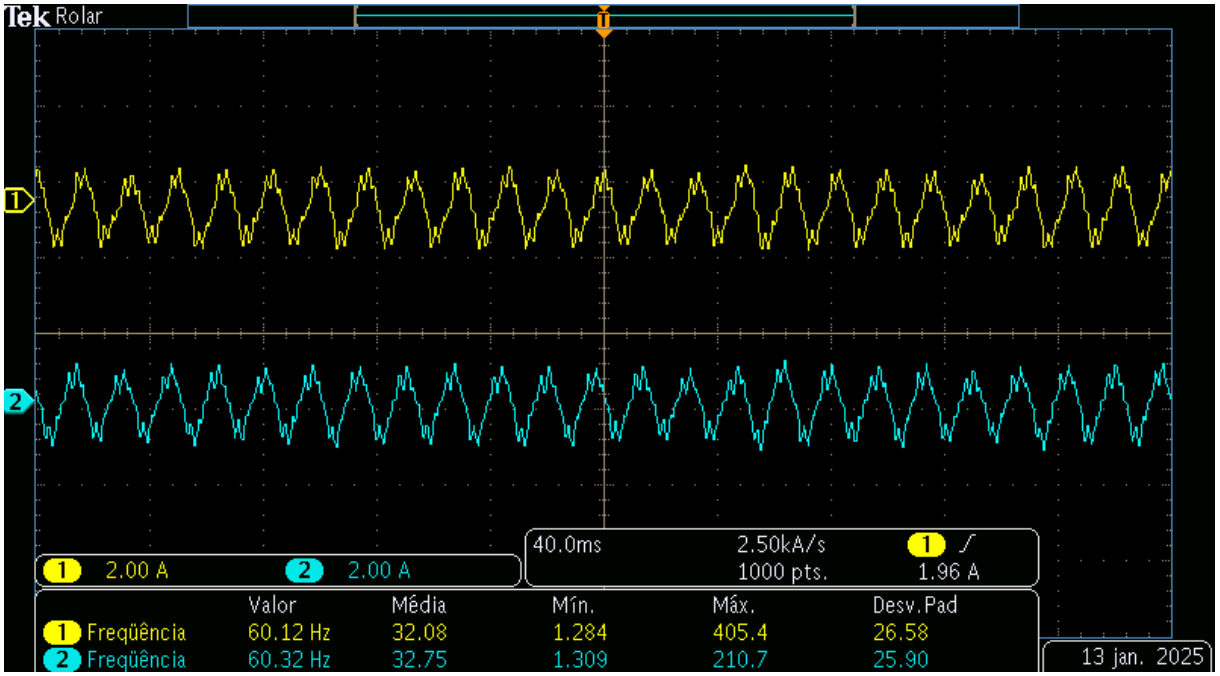
Em seguida, foram feitos testes com dois sinais de corrente. A Figura 28 e a Figura 29 ilustram, respectivamente, o funcionamento do supervisório e a medição do osciloscópio, para as duas curvas.

Figura 28 – Exibição no tempo de dois sinais de corrente.

Fonte: Elaborado pelo autor.

De maneira análoga, a análise realizada para um sinal de corrente, é possível perceber que os sinais exibidos no supervisório são semelhantes aos registrados no osciloscópio. Logo, é possível afirmar que a exibição feita pelo supervisório retrata corretamente os sinais interesse.

Figura 29 – Dois sinais de corrente medidos pelo osciloscópio.



Fonte: Elaborado pelo autor.

6 CONCLUSÃO

A implementação do sistema supervisório para monitoramento de sinais provenientes do DSP foi concluída com êxito, uma vez que foi possível utilizar o projeto proposto em um caso prático onde se observou a exibição de curvas que permitem avaliar o comportamento de variáveis escolhida e em diferentes formatos.

Contudo, algumas limitações foram identificadas e apontam para possíveis melhorias. A velocidade de transferência de dados pela comunicação SCI utilizada mostrou-se limitada, resultando na perda de informações, pois novos dados eram gerados enquanto informações anteriores ainda estavam sendo transmitidas. Uma solução para esse problema seria empregar *drivers* que suportem taxas de comunicação mais altas ou adotar protocolos mais rápidos, como SPI ou I2C.

Além disso, embora a taxa de atualização de um segundo tenha sido adequada para os objetivos iniciais, uma maior fluidez na exibição dos dados seria desejável, o que poderia ser alcançado com otimizações no software ou no método de visualização utilizado.

Assim, os resultados obtidos demonstram o potencial do sistema, mas evidenciam oportunidades para aprimoramentos futuros, visando maior eficiência e precisão no monitoramento.

REFERÊNCIAS

AXELSON, J. *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems*. 2. ed. Madison, WI 53704: Lakeview Research, 2007. ISBN 978-1931448-07-9.

COMPUTING, R. *PyQt5 5.15.11*. 2024. Disponível em: https://pypi.org/project/PyQt5/. Acesso em: 15 out. 2024.

COMPUTING, R. *What is PyQt?* 2025. Disponível em: https://www.riverbankcomputing.com/software/pyqt/. Acesso em: 04 jan. 2025.

FITZPATRICK, M. *Create GUI Applications with Python & Qt5 (PyQt5 Edition): The hands-on guide to making apps with Python*. [S.l.]: Martin Fitzpatrick, 2020.

FOROUZAN, B. A. *Data Communications and Networking*. 5. ed. New York: The McGraw-Hill Companies, Inc, 2013.

HAYKIN, S. *Digital Communication Systems*. Wiley, 2013. ISBN 9780471647355. Disponível em: <https://books.google.com.br-/books?id=YGZXAAAACAAJ>https://books.google.com.br/books?id=YGZXAAAACAAJ.

INSTRUMENTS, T. *TMS320F2833x, TMS320F2823x Real-Time Microcontrollers*. 2007. Disponível em: https://www.ti.com/lit/ds/symlink/tms320f28335.pdf. Acesso em: 02 dez. 2024.

INSTRUMENTS, T. *TMS320x2833x, 2823x Serial Communications Interface (SCI) Reference Guide*. 2008. Disponível em: http://mil.ufl.edu/4744/docs/TI_DSP/TI-docs/sprufz5a.pdf. Acesso em: 12 dez. 2024.

INSTRUMENTS, T. *TMS320x2833x, TMS320x2823x Technical Reference Manual*. 2020. Disponível em: https://www.ti.com/lit/ug/sprui07/sprui07.pdf. Acesso em: 05 nov. 2024.

INTERNATIONAL, F. T. D. *FT232R USB UART IC Datasheet*. 2020. Disponível em: https://ftdichip.com/wp-content/uploads/2020/08/DS_FT232R.pdf. Acesso em: 21 dez. 2024.

MANO, M.; CILETTI, M. *Digital Design: With an Introduction to the Verilog HDL*. Pearson, 2013. (Always learning). ISBN 9780273764526. Disponível em: <https://books.google.com.br-/books?id=bm9smAEACAAJ>https://books.google.com.br/books?id=bm9smAEACAAJ.

TIBOLA, L. R. *Geração de Sistemas Supervisórios a partir de Modelos Orientados a Objetos*. Dissertação (Mestrado) — Universidade Federal do Rio Grande do Sul, Porto Alegre, 2000.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e aplicações*. 11. ed. Rua Nelson Francisco, 26, Limão: Pearson Education do Brasil, 2011.

YU, X. et al. The new frontier of smart grids. *IEEE Industrial Electronics Magazine*, IEEE, v. 5, n. 3, p. 49–63, 2011.

Apêndices

APÊNDICE A – CÓDIGO-FONTE DO DSP

Listing A.1 – Código-fonte do algoritmo desenvolvido em C no CCS

```
//Funcoes SCI
void scia_echoback_init(void);
void scia_fifo_init(void);
void scia_xmit(int a);
void scia_msg(char *msg);

void numeroParaString(float numero, char* str);
interrupt void  adc_isr(void);

unsigned int adcInterruptCount1 = 0;

//Variaveis SCI
char *msg_aux = "\n\0";
char buffer1_ad[8];
char buffer2_ad[8];

float ang1 = 0, senoide1_ad = 0.0, senoide2_ad = 0.0, ad_freq =
    ↪ 0.0, ad_freq_aux = 2.0;

void main(void)
{
// Step 1. Initialize System Control:
// PLL, WatchDog, enable Peripheral Clocks
    InitSysCtrl();

// Step 2. Initalize GPIO:
// Init GPIO pins for ePWM1A, ePWM1B, ePWM2A, ePWM2B, ePWM3A e
    ↪ ePWM3B e Ecap1

    InitEPwmGpio();
    InitECap1Gpio();
    InitEQep2Gpio();

// Step 3. Clear all interrupts and initialize PIE vector table:
// Disable CPU interrupts
    DINT;
```

```
35
36 // Initialize the PIE control registers to their default state.
37    InitPieCtrl();
38
39 // Disable CPU interrupts and clear all CPU interrupt flags:
40    IER = 0x0000;
41    IFR = 0x0000;
42
43 // Initialize the PIE vector table with pointers to the shell
   ↪ Interrupt
44 // Service Routines (ISR).
45    InitPieVectTable();
46
47 // Interrupts are re-mapped to ISR functions found within this
   ↪ file.
48    EALLOW;
49    PieVectTable.ADCINT = &adc_isr;
50    EDIS;
51
52 // Step 4. Initialize A/D Peripheral:
53    InitAdc();
54
55    scia_fifo_init();        // Initialize the SCI FIFO
56    scia_echoback_init();   // Initalize SCI for echoback
57
58 //  Stop PWM counter timer ( TBCLKSYNC - Sincroniza os clocks e
   ↪ Parando todos os moduloas PWM)
59    EALLOW;
60    SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 0; // - reseta  TBCLKSYNC
61    EDIS;
62
63    InitEPwm1();
64
65 //  Start PWM counter timer. All PWM Channels are syncronized
66    EALLOW;
67    SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1;
68    EDIS;
69
70 // Step 5. User specific code, enable interrupts:
71    IER |=  M_INT1; //interrupcao do Timer0 & ADC
72
```

```
73 // Enable ADCINT in PIE
74    PieCtrlRegs.PIEIER1.bit.INTx6 = 1;  //Utilizar a interrupcao
      ↪ do ADC
75
76 // Enable global Interrupts and higher priority real-time debug
   ↪ events:
77    EINT;   // Enable Global interrupt INTM
78    ERTM;   // Enable Global realtime interrupt DBGM
79
80 // Step 6. IDLE loop.
81    ConfigGPIO();  // Configura os pinos IO
82
83 //Funcao que configura o ADC para realizar a conversao via
   ↪ interrupcao do PWM
84    ConfigCAD();
85
86     for(;;){
87
88         if(adcInterruptCount1 == 4){
89
90             adcInterruptCount1=0;
91
92             numeroParaString(senoide1_ad, buffer1_ad);
93             numeroParaString(senoide2_ad, buffer2_ad);
94
95             scia_msg(buffer1_ad);
96             scia_msg(msg_aux);
97             scia_msg(buffer2_ad);
98             scia_msg(msg_aux);
99
100        }
101    }
102
103 } //fim do main
104
105 interrupt void  adc_isr(void){
106     adcInterruptCount1++;
107
108     ad_freq = ad_freq_aux;
109     ang1 = ang1 + 6.2837e-004 * ad_freq;
110     if (ang1 > 6.283)  ang1 = ang1 - 6.283;
```

```
    if (ang1 < -6.28) ang1 = 6.283 + ang1;

    senoide1_ad = 100 * sin(ang1);
    senoide2_ad = 100 * sin(ang1 + 2.0944);
}

void ConfigGPIO(void)
{
    // Configura GPIO
    EALLOW;

    GpioCtrlRegs.GPAPUD.bit.GPIO28 = 0;    // Enable pull-up for GPIO28 (SCIRXDA)
    GpioCtrlRegs.GPAPUD.bit.GPIO29 = 0;    // Enable pull-up for GPIO29 (SCITXDA)

    // Configura os pinos GPIO28 e GPIO29 como SCI-A
    GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 1;  // SCITXDA
    GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 1;  // SCIRXDA

    EDIS;
}

void scia_echoback_init()
{

    SciaRegs.SCICCR.all =0x0007;   // 1 stop bit,  No loopback
                                   // No parity,8 char bits,
                                   // async mode, idle-line protocol
    SciaRegs.SCICTL1.all =0x0003;
    SciaRegs.SCICTL2.all =0x0003;
    SciaRegs.SCICTL2.bit.TXINTENA =1;
    SciaRegs.SCICTL2.bit.RXBKINTENA =1;
    #if (CPU_FRQ_150MHZ)
        SciaRegs.SCIHBAUD = 0x0000;
        SciaRegs.SCILBAUD = 0x0004; // 921600 baud @LSPCLK = 37.5 MHz
    #endif

    SciaRegs.SCICTL1.all =0x0023;  // Relinquish SCI from Reset
```

```
148 }
149
150 // Transmit a character from the SCI
151 void scia_xmit(int a)
152 {
153     while (SciaRegs.SCIFFTX.bit.TXFFST != 0) {}
154     SciaRegs.SCITXBUF=a;
155 }
156
157 void scia_msg(char * msg)
158 {
159     int i = 0;
160     while(msg[i] != '\0')
161     {
162         scia_xmit(msg[i]);
163
164         i++;
165     }
166 }
167
168 // Initalize the SCI FIFO
169 void scia_fifo_init()
170 {
171     SciaRegs.SCIFFTX.all=0xE040;
172     SciaRegs.SCIFFRX.all=0x204f;
173     SciaRegs.SCIFFCT.all=0x0;
174
175 }
176
177 void numeroParaString(float numero, char *str) {
178     int posicao_buffer = 0;
179
180     if(numero < 0){
181       numero *= -1;
182       str[posicao_buffer] = '-';
183       posicao_buffer++;
184     }
185
186     int parteInteira = (int)numero;
187     float parteDecimal = numero - parteInteira;
188
```

```
189      // Converte a parte inteira para string
190     if (parteInteira == 0) {
191         str[posicao_buffer++] = '0';
192     } else {
193         int temp = parteInteira;
194         int numDigitos = 0;
195         // Conta o numero de digitos na parte inteira
196         while (temp > 0) {
197             temp /= 10;
198             numDigitos++;
199         }
200 
201         // Escreve os digitos da parte inteira no buffer
202         temp = parteInteira;
203         int i;
204         for (i = numDigitos - 1; i >= 0 && posicao_buffer < 7; i
            ↪ --) {
205             str[posicao_buffer + i] = (temp % 10) + '0';
206             temp /= 10;
207         }
208         posicao_buffer += numDigitos;
209     }
210 
211     if (parteDecimal != 0){
212         // Adiciona o ponto decimal, se ainda houver espaco
213         if (posicao_buffer < 6) {
214             str[posicao_buffer++] = '.';
215         }
216         int i = 0;
217         for (i = 0; i < 4 && posicao_buffer < 7; i++) {
218             parteDecimal *= 10;
219             int digit = (int)parteDecimal;
220             if (digit == 0 && parteDecimal == 0.0) {  // Para
                ↪ evitar zeros desnecessarios
221                 str[posicao_buffer++] = '0';
222                 break;
223             }
224             str[posicao_buffer++] = digit + '0';
225             parteDecimal -= digit;
226         }
227     }
```

```
228
229     // Adiciona o caractere nulo para terminar a string
230     str[posicao_buffer] = '\0';
231 }
```

APÊNDICE B – CÓDIGO-FONTE DA INTERFACE GRÁFICA DO SUPERVISÓRIO

Listing B.1 – Código-fonte da interface desenvolvida com Qt*Designer*

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>MainWindow</class>
 <widget class="QMainWindow" name="MainWindow">
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>1360</width>
    <height>935</height>
   </rect>
  </property>
  <property name="windowTitle">
   <string>MainWindow</string>
  </property>
  <widget class="QWidget" name="centralwidget">
   <widget class="QGroupBox" name="groupBox">
    <property name="geometry">
     <rect>
      <x>40</x>
      <y>80</y>
      <width>1281</width>
      <height>151</height>
     </rect>
    </property>
    <property name="title">
     <string>Configura  es</string>
    </property>
    <widget class="QWidget" name="gridLayoutWidget_2">
     <property name="geometry">
      <rect>
       <x>400</x>
       <y>30</y>
       <width>261</width>
```

```
35          <height>94</height>
36         </rect>
37        </property>
38        <layout class="QGridLayout" name="gridLayout_plotTemp">
39         <item row="0" column="0">
40          <widget class="QLabel" name="label_PlotTempo">
41           <property name="text">
42            <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p align
              ↪ =&quot;center&quot;&gt;&lt;span style=&quot; font-
              ↪ size:12pt; font-weight:696;&quot;&gt;Plotagem no
              ↪ tempo&lt;/span&gt;&lt;/p&gt;&lt;/body&gt;&lt;/html&
              ↪ gt;</string>
43           </property>
44          </widget>
45         </item>
46         <item row="2" column="0">
47          <layout class="QHBoxLayout" name="horizontalLayout_5">
48           <item>
49            <spacer name="horizontalSpacer_2">
50             <property name="orientation">
51              <enum>Qt::Horizontal</enum>
52             </property>
53             <property name="sizeHint" stdset="0">
54              <size>
55               <width>128</width>
56               <height>20</height>
57              </size>
58             </property>
59            </spacer>
60           </item>
61           <item>
62            <widget class="QPushButton" name="pushButton_plotTemp">
63             <property name="text">
64              <string>Confirme</string>
65             </property>
66            </widget>
67           </item>
68          </layout>
69         </item>
70         <item row="1" column="0">
71          <layout class="QHBoxLayout" name="horizontalLayout_2">
```

```
72          <item>
73           <widget class="QLabel" name="label_4">
74            <property name="text">
75             <string>N mero de sinais</string>
76            </property>
77           </widget>
78          </item>
79          <item>
80           <widget class="QComboBox" name="comboBox_plotTemp"/>
81          </item>
82         </layout>
83        </item>
84       </layout>
85      </widget>
86      <widget class="QWidget" name="gridLayoutWidget">
87       <property name="geometry">
88        <rect>
89         <x>50</x>
90         <y>30</y>
91         <width>281</width>
92         <height>96</height>
93        </rect>
94       </property>
95       <layout class="QGridLayout" name="gridLayout">
96        <item row="1" column="0">
97         <layout class="QHBoxLayout" name="horizontalLayout_4">
98          <item>
99           <widget class="QLabel" name="label_portaComSer">
100           <property name="text">
101            <string>Porta</string>
102           </property>
103          </widget>
104         </item>
105         <item>
106          <widget class="QComboBox" name="comboBox_portas"/>
107         </item>
108         <item>
109          <layout class="QHBoxLayout" name="horizontalLayout_6">
110           <item>
111            <widget class="QLabel" name="label_baudrate">
112             <property name="text">
```

```
113                <string>Baudrate:</string>
114               </property>
115              </widget>
116             </item>
117             <item>
118              <widget class="QLabel" name="label">
119               <property name="text">
120                <string>921600</string>
121               </property>
122              </widget>
123             </item>
124            </layout>
125           </item>
126          </layout>
127         </item>
128         <item row="0" column="0">
129          <widget class="QLabel" name="label_selecInicial">
130           <property name="text">
131            <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p align
               ↪ =&quot;center&quot;&gt;&lt;span style=&quot; font-
               ↪ size:12pt; font-weight:696;&quot;&gt;Sele  o
               ↪ Inicial&lt;/span&gt;&lt;/p&gt;&lt;/body&gt;&lt;/
               ↪ html&gt;</string>
132           </property>
133          </widget>
134         </item>
135         <item row="2" column="0">
136          <layout class="QHBoxLayout" name="horizontalLayout_8">
137           <item>
138            <layout class="QHBoxLayout" name="horizontalLayout">
139             <item>
140              <widget class="QLabel" name="label_status">
141               <property name="text">
142                <string>Status:</string>
143               </property>
144              </widget>
145             </item>
146             <item>
147              <widget class="QLabel" name="label_alteraStatus">
148               <property name="text">
149                <string>Status</string>
```

```
150             </property>
151            </widget>
152           </item>
153          </layout>
154         </item>
155         <item>
156          <spacer name="horizontalSpacer">
157           <property name="orientation">
158            <enum>Qt::Horizontal</enum>
159           </property>
160           <property name="sizeHint" stdset="0">
161            <size>
162             <width>40</width>
163             <height>20</height>
164            </size>
165           </property>
166          </spacer>
167         </item>
168         <item>
169          <widget class="QPushButton" name="pushButton_selInicial"
               ↪ >
170           <property name="text">
171            <string>Confirme</string>
172           </property>
173          </widget>
174         </item>
175        </layout>
176       </item>
177      </layout>
178     </widget>
179     <widget class="QPushButton" name="pushButton_iniciar">
180      <property name="geometry">
181       <rect>
182        <x>1160</x>
183        <y>60</y>
184        <width>91</width>
185        <height>61</height>
186       </rect>
187      </property>
188      <property name="text">
189       <string>Iniciar</string>
```

```
190      </property>
191     </widget>
192     <widget class="QWidget" name="layoutWidget">
193      <property name="geometry">
194       <rect>
195        <x>670</x>
196        <y>70</y>
197        <width>147</width>
198        <height>53</height>
199       </rect>
200      </property>
201      <layout class="QGridLayout" name="gridLayout_3">
202       <item row="0" column="0">
203        <layout class="QHBoxLayout" name="horizontalLayout_3">
204         <item>
205          <widget class="QLabel" name="label_2">
206           <property name="text">
207            <string>Zoom horizontal (%):</string>
208           </property>
209          </widget>
210         </item>
211         <item>
212          <widget class="QLabel" name="label_horizontalSlider">
213           <property name="text">
214            <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p&gt
               ↪ ;-&lt;/p&gt;&lt;/body&gt;&lt;/html&gt;</string>
215           </property>
216          </widget>
217         </item>
218        </layout>
219       </item>
220       <item row="1" column="0">
221        <widget class="QSlider" name="horizontalSlider">
222         <property name="orientation">
223          <enum>Qt::Horizontal</enum>
224         </property>
225        </widget>
226       </item>
227      </layout>
228     </widget>
229     <widget class="QWidget" name="layoutWidget">
```

```
230      <property name="geometry">
231       <rect>
232        <x>860</x>
233        <y>30</y>
234        <width>251</width>
235        <height>63</height>
236       </rect>
237      </property>
238      <layout class="QGridLayout" name="gridLayout_2">
239       <item row="1" column="0">
240        <layout class="QHBoxLayout" name="horizontalLayout_11">
241         <item>
242          <spacer name="horizontalSpacer_3">
243           <property name="orientation">
244            <enum>Qt::Horizontal</enum>
245           </property>
246           <property name="sizeHint" stdset="0">
247            <size>
248             <width>40</width>
249             <height>20</height>
250            </size>
251           </property>
252          </spacer>
253         </item>
254         <item>
255          <widget class="QPushButton" name="
               ↪ pushButton_plotLissajous">
256           <property name="text">
257            <string>Confirme</string>
258           </property>
259          </widget>
260         </item>
261         <item>
262          <spacer name="horizontalSpacer_4">
263           <property name="orientation">
264            <enum>Qt::Horizontal</enum>
265           </property>
266           <property name="sizeHint" stdset="0">
267            <size>
268             <width>40</width>
269             <height>20</height>
```

```
270            </size>
271           </property>
272          </spacer>
273         </item>
274        </layout>
275       </item>
276       <item row="0" column="0">
277        <widget class="QLabel" name="label_plotLissajous">
278         <property name="text">
279          <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p align
             ↪ =&quot;center&quot;&gt;&lt;span style=&quot; font-
             ↪ size:12pt; font-weight:600;&quot;&gt;Lissajous&lt;/
             ↪ span&gt;&lt;/p&gt;&lt;/body&gt;&lt;/html&gt;</
             ↪ string>
280         </property>
281        </widget>
282       </item>
283      </layout>
284     </widget>
285    </widget>
286    <widget class="QWidget" name="widget_1" native="true">
287     <property name="geometry">
288      <rect>
289       <x>40</x>
290       <y>250</y>
291       <width>1281</width>
292       <height>611</height>
293      </rect>
294     </property>
295     <property name="styleSheet">
296      <string notr="true">background-color:rgb(0,0,0);</string>
297     </property>
298    </widget>
299    <widget class="QLabel" name="label_titulo">
300     <property name="geometry">
301      <rect>
302       <x>30</x>
303       <y>20</y>
304       <width>1291</width>
305       <height>51</height>
306      </rect>
```

```
307     </property>
308     <property name="text">
309      <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p&gt;&lt;
         ↪ span style=&quot; font-size:16pt; font-weight:696;&quot
         ↪ ;&gt;Supervis rio para sinais do DSP&lt;/span&gt;&lt;/
         ↪ p&gt;&lt;/body&gt;&lt;/html&gt;</string>
310     </property>
311     <property name="alignment">
312      <set>Qt::AlignCenter</set>
313     </property>
314    </widget>
315    <widget class="QFrame" name="frame">
316     <property name="geometry">
317      <rect>
318       <x>1390</x>
319       <y>330</y>
320       <width>211</width>
321       <height>131</height>
322      </rect>
323     </property>
324     <property name="frameShape">
325      <enum>QFrame::StyledPanel</enum>
326     </property>
327     <property name="frameShadow">
328      <enum>QFrame::Raised</enum>
329     </property>
330    </widget>
331   </widget>
332   <widget class="QMenuBar" name="menubar">
333    <property name="geometry">
334     <rect>
335      <x>0</x>
336      <y>0</y>
337      <width>1360</width>
338      <height>26</height>
339     </rect>
340    </property>
341    <widget class="QMenu" name="menuMenu_Inical">
342     <property name="title">
343      <string>Menu Inical</string>
344     </property>
```

```
345      <addaction name="actionReiniciar"/>
346      <addaction name="separator"/>
347      <addaction name="actionFechar_2"/>
348     </widget>
349     <addaction name="menuMenu_Inical"/>
350    </widget>
351    <widget class="QStatusBar" name="statusbar"/>
352    <action name="actionReiniciar">
353     <property name="text">
354      <string>Reiniciar</string>
355     </property>
356    </action>
357    <action name="actionFechar">
358     <property name="text">
359      <string>Fechar</string>
360     </property>
361    </action>
362    <action name="actionFechar_2">
363     <property name="text">
364      <string>Fechar</string>
365     </property>
366    </action>
367   </widget>
368   <resources/>
369   <connections/>
370  </ui>
```

APÊNDICE C – CÓDIGO-FONTE DA APLICAÇÃO

Listing C.1 – Código Python da aplicação gráfica com PyQt5

```
import sys
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
from PyQt5.QtCore import QThread, pyqtSignal, QMutex,  QMutex,
    ↪ QTimer
from PyQt5 import uic
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
import serial.tools.list_ports
import serial
from collections import deque
import pyqtgraph as pg
import numpy as np

class MyGUI(QMainWindow):
    def __init__(self):
        super(MyGUI, self).__init__()
        uic.loadUi("gui.ui", self)

        #Centraliza a GUI no meio da tela ao executar
        screen = QApplication.primaryScreen()
        screen_geometry = screen.availableGeometry()  # Obtem a
            ↪ geometria da tela disponivel
        # Calcula a posicao centralizada
        screen_width = screen_geometry.width()
        screen_height = screen_geometry.height()
        # Obtem as dimensoes da janela
        window_width = self.width()
        window_height = self.height()
        # Calcula as coordenadas para centralizar a janela
        x = (screen_width - window_width) // 2
        y = (screen_height - window_height) // 2
        # Move a janela para a posicao central
        self.move(x, y)
```

```
33
34          self.actionReiniciar.triggered.connect(self.reiniciar)
35          self.actionFechar_2.triggered.connect(self.fechar)
36
37          #Variaveis para controle de eventos
38          self.escolha_plot = '1'
39          self.escolha_lissajous = '1'
40          self.ultimo_botao_press = '0'
41          self.porcentagem_corte = 0
42          self.faixa_inicial = 0.0
43          self.faixa_final = 0.0
44          self.pre_trigger_1 = 0.0
45          self.pre_trigger_22 = 0.0
46          #Variaveis para controle da comunicacao serial
47          self.ser = None
48          #Outras variaveis
49          self.mutex = QMutex()
50          self.data_thread = None
51          #Frequencia de amostragem dos dados
52          self.freq = 10000/4
53          self.sample_rate = 1/self.freq
54
55          # Cria um objeto GraphicsLayoutWidget do PyQtGraph
56          self.graphics_layout_widget = pg.GraphicsLayoutWidget()
57
58          # Layout vertical para inserir o GraphicsLayoutWidget no
               ↪ widget
59          self.widget_layout = QVBoxLayout(self.widget_1)
60          self.widget_layout.addWidget(self.graphics_layout_widget)
61
62          # Conecta os botoes aos metodos
63          self.pushButton_selInicial.clicked.connect(self.
               ↪ configure_selInicial)
64          self.pushButton_plotTemp.clicked.connect(self.
               ↪ configure_plotTemp)
65          self.pushButton_plotLissajous.clicked.connect(self.
               ↪ configure_plotLissajous)
66          self.pushButton_iniciar.clicked.connect(self.
               ↪ ajusta_aquisicao)
67
68          #  Lista as portas seriais disponiveis
```

```
69          portas = serial.tools.list_ports.comports()
70          for porta in portas:
71              self.comboBox_portas.addItem(porta.device)
72
73          #Conecta as opcoes de comboBox da plotagem no tempo
74          self.comboBox_plotTemp.addItem(" ")
75          self.comboBox_plotTemp.addItem("1")
76          self.comboBox_plotTemp.addItem("2")
77
78          # Conectando o sinal de mudanca de selecao ao metodo
79          self.comboBox_plotTemp.currentIndexChanged.connect(self.
            ↪ seleciona_sinais)
80
81          #self.horizontalSlider.setMinimum(0)
82          self.horizontalSlider.setRange(0,9)
83          self.horizontalSlider.setValue(0)
84          self.horizontalSlider.setTickInterval(1)
85          self.label_horizontalSlider.setText("0")
86          self.horizontalSlider.valueChanged.connect(self.
            ↪ horizontalZoom)
87
88          self.show()
89
90      """
91      Configuracao do Menu
92      """
93      # Botao para reiniciar aplicacao
94      def reiniciar(self):
95          reinicia_app()
96
97      # Botao para fehcar aplicacao
98      def fechar(self):
99          self.close()
100
101     # Configuracoes da "Selecao Inicial"
102     def configure_selInicial(self):
103         if self.ser and self.ser.is_open:
104             self.ser.close()
105             self.ser = None
106             self.pushButton_selInicial.setText("Conecta")
107             self.label_alteraStatus.setText("Desconectado")
```

```
108                 if self.ser:
109                     self.ser.quit()
110                     self.ser = None
111             else:
112                 porta_selecionada = self.comboBox_portas.currentText
                    ↪ ()
113                 baud_rate = 921600
114                 try:
115                     self.ser = serial.Serial(porta_selecionada,
                        ↪ baud_rate)
116                     if self.ser.is_open:
117                         self.pushButton_selInicial.setText("
                            ↪ Desconectar")
118                         self.label_alteraStatus.setText("Conectado")
119
120                     else:
121                         self.label_alteraStatus.setText("Erro ao
                            ↪ Conectar")
122                 except Exception as e:
123                     self.serial_connection = None
124                     QtWidgets.QMessageBox.critical(self, "Error", f"
                        ↪ Fail: {str(e)}")
125
126     # Ajusta o zoom horizontal e atualiza o label com o valor
        ↪ correspondente.
127     def horizontalZoom(self, value_slider):
128         self.porcentagem_corte = value_slider * 5
129         self.label_horizontalSlider.setText(f"{self.
            ↪ porcentagem_corte*2}")
130
131     # Configuracao do grafico para um sinal em funcao do tempo
132     def configure_plot_1(self):
133         self.plot1 = self.graphics_layout_widget.addPlot(row=0,
            ↪ col=0)
134         self.plot1.showGrid(x=True, y=True)
135         self.curve1 = self.plot1.plot(pen='r')
136
137     # Configuracao dos graficos para dois sinais em funcao do
        ↪ tempo
138     def configure_plot_2(self):
```

```
139         self.plot1 = self.graphics_layout_widget.addPlot(row=0,
            ↪ col=0)
140         self.plot1.showGrid(x=True, y=True)
141         self.curve1 = self.plot1.plot(pen='r')
142 
143         self.plot2 = self.graphics_layout_widget.addPlot(row=1,
            ↪ col=0)
144         self.plot2.showGrid(x=True, y=True)
145         self.curve2 = self.plot2.plot(pen='b')
146 
147     # Configuracao do grafico no formato Lissajous
148     def configure_plot_5(self):
149         self.plot1 = self.graphics_layout_widget.addPlot(row=0,
            ↪ col=0)
150         self.curve1 = self.plot1.plot(pen='r')
151         print("Cheguei configure_plot_5")
152 
153     # Obtem a opcao selecionada no comboBox de exibicao em funcao
        ↪  do tempo
154     def seleciona_sinais(self):
155         self.escolha_plot = self.comboBox_plotTemp.currentText()
156         print(f'Selecionado: {self.escolha_plot}')
157 
158     # Configura os deques de acordo com a quantidade de sinais a
        ↪ serem exibidos em funcao do tempo
159     def configure_plotTemp(self):
160         self.eixo_x = deque()
161         if (self.escolha_plot == '2'):
162             self.configure_plot_2()
163             self.eixo_y1 = deque()
164             self.eixo_y2 = deque()
165         elif (self.escolha_plot == '1'):
166             self.configure_plot_1()
167             self.eixo_y1 = deque()
168             self.eixo_y2 = None
169         self.ultimo_botao_press = '0'
170 
171     # Configura os deques de acordo com a exibicao no formato de
        ↪ Lissajous
172     def configure_plotLissajous(self):
173         self.eixo_x = deque()
```

```
174         if (self.escolha_lissajous == '1'):
175             self.configure_plot_5()
176             self.eixo_y1 = deque()
177             self.eixo_y2 = deque()
178 
179         self.ultimo_botao_press = '1'
180 
181     # Ajusta a configuracao do plot e inicia a thread de
        ↪ aquisicao de dados
182     def ajusta_aquisicao(self):
183         self.ajusta_plot()
184         if not self.data_thread:
185             self.data_thread = DataAcquisitionThread(self.ser,
                ↪ self.mutex, self.eixo_x, self.eixo_y1, self.
                ↪ eixo_y2, self.sample_rate)
186             self.disparo_aquisicao()
187         else:
188             pass
189 
190     # Ajusta o tipo de plot com base na selecao feita
191     def ajusta_plot(self):
192         if self.ultimo_botao_press == '1':
193             if self.escolha_lissajous == '1':
194                 self.ultima_escolha_plot = '5'
195         else:
196             self.ultima_escolha_plot = self.escolha_plot
197         print(f'Escolha foi:{self.ultima_escolha_plot}')
198 
199     # Inicia a aquisicao de dados e configura a atualizacao do
        ↪ plot a cada 1 segundo
200     def disparo_aquisicao(self):
201         self.data_thread.nova_quant_sinais.emit(self.
            ↪ ultima_escolha_plot)
202         self.data_thread.start()
203 
204         # Iniciar o timer de atualizacao
205         self.update_timer = QTimer(self)
206         self.update_timer.timeout.connect(self.update_plot)
207         self.update_timer.start(1000)
208 
209     # Atualiza os graficos
```

```
210     def update_plot(self):
211
212         self.mutex.lock()
213
214         x_vals = list(self.eixo_x)
215         y_vals1 = list(self.eixo_y1)
216
217         if self.eixo_x:
218             self.eixo_x.clear()
219
220         if self.eixo_y1:
221             self.eixo_y1.clear()
222
223         if (self.ultima_escolha_plot == '2'
224             or self.ultima_escolha_plot == '5'):
225
226             y_vals2 = list(self.eixo_y2)
227             self.eixo_y2.clear()
228
229         self.mutex.unlock()
230
231         #Atualizacao do espaco horizontal da curva conforme
            ↪ atualizacao no slider
232         self.faixa_inicial = int((self.porcentagem_corte / 100) *
            ↪  len(x_vals))
233         self.faixa_final = int(len(x_vals) - (self.
            ↪ porcentagem_corte/ 100) * len(x_vals))
234
235         x_vals = x_vals[self.faixa_inicial:self.faixa_final]
236
237         if (self.ultima_escolha_plot == '1'):
238             y_vals1 = y_vals1[self.faixa_inicial:self.faixa_final
                ↪ ]
239
240             if (len(x_vals) == len(y_vals1)) and len(x_vals) !=
                ↪ 0:
241                 self.curve1.setData(x_vals, y_vals1)
242             else:
243                 print("[Plot 1] Vetores de dados com tamnhos
                    ↪ diferentes!")
244
```

```
245         if (self.ultima_escolha_plot == '2'):
246             y_vals1 = y_vals1[self.faixa_inicial:self.faixa_final
                ↪ ]
247
248             if (len(x_vals) == len(y_vals1)) and len(x_vals) !=
                ↪ 0:
249                 self.curve1.setData(x_vals, y_vals1)
250             else:
251                 print("[Plot 2.1] Vetores de dados com tamnhos
                    ↪ diferentes!")
252
253             y_vals2 = y_vals2[self.faixa_inicial:self.faixa_final
                ↪ ]
254
255             if (len(x_vals) == len(y_vals2)) and len(x_vals) !=
                ↪ 0:
256                 self.curve2.setData(x_vals, y_vals2)
257             else:
258                 print("[Plot 2.2] Vetores de dados com tamnhos
                    ↪ diferentes!")
259
260         if (self.ultima_escolha_plot == '5'):
261             self.curve1.setData(y_vals1, y_vals2)
262
263 """
264 Thread para aquisicao e processamento de dados seriais, com
    ↪ suporte a um ou dois sinais simultaneos.
265 """
266 class DataAcquisitionThread(QThread):
267     nova_quant_sinais = pyqtSignal(str)
268     data_ready = pyqtSignal(list)
269     print("Thread iniciada")
270
271     # Inicializa a thread com parametros de comunicacao, buffers
        ↪ e taxa de amostragem
272     def __init__(self, comSer, mutex, eixo_x, eixo_y1, eixo_y2,
        ↪ sample_rate):
273         super().__init__()
274         #Variaveis compartilhadas via argumento
275         self.comSer = comSer
276         self.mutex = mutex
```

```
277         self.eixo_x = eixo_x
278         self.eixo_y1 = eixo_y1
279         self.eixo_y2 = eixo_y2
280         self.sample_rate = sample_rate
281         #Variaveis compartilhadas via sinal
282         self.quant_sinais = "1"
283         self.nova_quant_sinais.connect(self.set_quant_sinais)
284         #Variaveis locais
285         self.ativo = True
286         self.buffer = ""
287         # Buffers locais
288         self.buffer_x = []
289         self.buffer_y1 = []
290         self.buffer_y2 = []
291         self.next_x = 0.0
292         # Controle para alternancia completa
293         self.ready_to_update_x = False
294         self.current_buffer = self.buffer_y1
295         self.pending_pair = None
296         #Controle do buffer_x quando so se tem 1 sinal
297         self.controleX_1sinal = False
298 
299     # Define a quantidade de sinais a ser adquirida e atualiza o
        ↪ estado da thread
300     def set_quant_sinais(self, quant_sinais):
301         self.quant_sinais = quant_sinais
302         print(f'Quantidade de sinais mudou: {quant_sinais}')
303         print(f"self.ativo = {self.ativo}")
304 
305     # Adquire e processa dados via comunicacao serial,
        ↪ armazenando-os nos buffers.
306     def run(self):
307 
308         '''
309         Aguarda um periodo para estabilizar a comunicacao.
310         '''
311         for _ in range(2):
312             a = self.comSer.readline()
313             print(f"[TA] Duas primeiras leituras: {a}")
314 
315         while self.ativo:
```

```
316
317             if self.quant_sinais == '2' or self.quant_sinais == '
                ↪ 5':
318
319                 # Leitura de dois valores, cada um em uma linha
320                 char = self.comSer.read(1).decode()  # Leia um
                    ↪ caractere
321                 self.buffer += char
322
323                 if char == '\n':  # Processa um dado completo ao
                    ↪ encontrar '\n'
324                     try:
325                         value = float(self.buffer.strip())  #
                            ↪ Converte para float
326                     except ValueError:
327                         print(f"[ERRO] Dado invalido recebido: {
                            ↪ self.buffer.strip()}")
328                         self.buffer = ""
329
330                         # Descartar par atual
331                         self.pending_pair = None
332                         self.current_buffer = self.buffer_y1
333                         continue
334
335                     if self.pending_pair is None:
336                         # Primeiro dado do par
337                         self.pending_pair = value
338                         self.current_buffer = self.buffer_y2
339                     else:
340                         # Segundo dado do par
341                         self.buffer_y1.append(self.pending_pair)
342                         self.buffer_y2.append(value)
343
344                         # Atualiza eixo_x somente apos o par ser
                            ↪ valido
345                         self.buffer_x.append(self.next_x)
346                         self.next_x = round(self.next_x + self.
                            ↪ sample_rate, 9)
347
348                         self.pending_pair = None
349                         self.current_buffer = self.buffer_y1
```

```
350
351                     # Reseta o buffer de leitura
352                     self.buffer = ""
353
354                 # Transferencia periodica para os deques
355                 if len(self.buffer_x) >= 200:
356                     self._transfer_to_deques()
357
358             else:
359                 # Leitura de um unico valor
360                 char = self.comSer.read(1).decode()  # Leia um
                    ↪ caractere
361                 self.buffer += char
362
363                 if char == '\n':  # Processa o dado completo ao
                    ↪ encontrar '\n'
364                     try:
365                         value = float(self.buffer.strip())  #
                            ↪ Converte para float
366                         self.controleX_1sinal = True
367                     except ValueError:
368                         print(f"[ERRO] Dado invalido recebido: {
                            ↪ self.buffer.strip()}")
369                         self.buffer = ""
370                         continue
371
372                     if self.controleX_1sinal == True:
373                         self.buffer_y1.append(value)
374
375                         self.buffer_x.append(self.next_x)
376                         self.next_x = round(self.next_x + self.
                            ↪ sample_rate, 9)
377                         self.controleX_1sinal = False
378
379                     # Reseta o buffer de leitura
380                     self.buffer = ""
381
382                 # Transferencia periodica para os deques
383                 if len(self.buffer_x) >= 200:
384                     self._transfer_to_deques()
385
```

```
    #Transfere dados dos buffers locais para os deques compartilhados.
    def _transfer_to_deques(self):
        self.mutex.lock()
        try:
            if self.quant_sinais == '2' or self.quant_sinais == '5':
                self.eixo_x.extend(self.buffer_x)
                self.eixo_y1.extend(self.buffer_y1)
                self.eixo_y2.extend(self.buffer_y2)
            else:
                self.eixo_x.extend(self.buffer_x)
                self.eixo_y1.extend(self.buffer_y1)
        finally:
            self.mutex.unlock()

        # Limpa os buffers locais apos a transferencia
        self.buffer_x.clear()
        self.buffer_y1.clear()
        self.buffer_y2.clear()

# Reinicia a aplicacao fechando e recriando a janela principal.
def reinicia_app():
    global window
    window.close()  # Fecha a janela atual
    window = MyGUI() # Cria uma nova instancia da janela principal

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MyGUI()
    sys.exit(app.exec_())
```